

Trabajo de Fin de Grado

Ingeniería en Tecnologías Industriales

Diseño e implementación de un sistema distribuido basado en ROS 2 y micro-ROS para el control de un motor BLDC

MEMORIA

Autor/a: Marc Ros Gazo
Director/a: Alba Pérez
Convocatoria: Enero 2026



Escola Tècnica Superior
d'Enginyeria Industrial de Barcelona

Resum

Aquest Treball de Fi de Grau s'ha realitzat a la Politechnika Poznanska (Universitat Tecnològica de Poznan) amb l'objectiu inicial de controlar una roda de reacció (reaction wheel) per a nanosatèl·lits. L'evolució del treball ha derivat en el disseny, la integració i la validació d'un sistema complet per al control d'un motor BLDC, integrant microcontroladors dins l'ecosistema ROS 2 mitjançant l'ús de micro-ROS.

El nucli del projecte es basa en una arquitectura de hardware composta per dos sistemes que operen de manera coordinada. D'una banda, una placa de desenvolupament (STM32 NUCLEO-F446ZE) actua com a unitat de processament principal; aquest dispositiu executa l'agent de micro-ROS i gestiona la comunicació amb l'ordinador central. De l'altra, s'utilitza el driver B-G431B-ESC1, encarregat exclusivament de la commutació electrònica i l'accionament físic del motor. La connexió entre tots dos dispositius s'ha resolt mitjançant la comunicació sèrie (UART), fet que permet que la placa principal envii ordres i rebi informació del driver en temps real.

A nivell de firmware, s'ha desenvolupat la lògica necessària per rebre consignes de velocitat des de ROS 2, processar-les en el microcontrolador principal i enviar-les al controlador del motor, tot validant per separat el correcte transport de les dades en cada etapa. Aquest desenvolupament ha permès establir un pont eficaç entre el programari de robòtica i el hardware d'actuació.

Finalment, el treball abasta la caracterització dinàmica del sistema. Mitjançant assajos experimentals i simulació amb Simulink, s'ha modelat la resposta del conjunt motor-actuador i s'ha dissenyat un controlador PID específic. Gràcies a això, s'ha pogut proposar una arquitectura de control final on el microcontrolador principal s'encarrega de la dinàmica del motor en la seva totalitat, tancant el llaç de velocitat mitjançant la lectura de sensors externs.

Resumen

Este Trabajo de Fin de Grado se ha realizado en la Politechnika Poznanska (Universidad Tecnológica de Poznan) con el objetivo inicial de controlar una rueda de reacción (reaction wheel) para nanosatélites. La evolución del trabajo ha derivado en el diseño, integración y validación de un sistema completo para el control de un motor BLDC, integrando microcontroladores dentro del ecosistema ROS 2 mediante el uso de micro-ROS.

El núcleo del proyecto se basa en una arquitectura de hardware compuesta por dos sistemas que operan de forma coordinada. Por un lado, una placa de desarrollo (STM32 NUCLEO-F446ZE) actúa como la unidad de procesamiento principal; este dispositivo ejecuta el agente de micro-ROS y gestiona la comunicación con el ordenador central. Por otro lado, se utiliza el driver B-G431B-ESC1, encargado exclusivamente de la conmutación electrónica y el accionamiento físico del motor. La conexión entre ambos dispositivos se ha podido resolver mediante la comunicación serie (UART), permitiendo que la placa principal envíe comandos y información del driver en tiempo real.

A nivel de firmware, se ha desarrollado la lógica necesaria para recibir consignas de velocidad desde la ROS 2, procesarlas en el microcontrolador principal y enviarlas al controlador del motor, validando por separado el correcto transporte de los datos en cada etapa. Este desarrollo ha permitido establecer un puente eficaz entre el software de robótica y el hardware de actuación.

Finalmente, el trabajo abarca la caracterización dinámica del sistema. Mediante ensayos experimentales y simulación en Simulink, se ha modelado la respuesta del conjunto motor-actuador y se ha diseñado un controlador PID específico. Gracias a esto, se ha podido proponer una arquitectura de control final donde el microcontrolador principal se encargue de la dinámica del motor en su totalidad, cerrando el lazo de velocidad usando la lectura de sensores externos.

Abstract

This Bachelor's Thesis, carried out at the Politechnika Poznanska (Poznan University of Technology) with the initial objective of controlling a reaction wheel for nanosatellites, presents the design, integration, and validation of a complete control system for a BLDC motor, integrating microcontrollers into the ROS 2 ecosystem using micro-ROS.

The core of the project is based on a hardware architecture composed of two systems operating in coordination. On one hand, an STM32 NUCLEO-F446ZE development board acts as the main processing unit; this device runs the micro-ROS agent and manages communication with the central computer. On the other hand, the B-G431B-ESC1 driver is used, tasked exclusively with the electronic commutation and physical actuation of the motor. The connection between both devices has been resolved via serial communication (UART), allowing the main board to send commands and receive driver information in real-time.

At the firmware level, the necessary logic has been developed to receive speed setpoints from ROS 2, process them in the main microcontroller, and forward them to the motor controller, validating the correct transport of data at each stage. This development has established an effective bridge between high-level robotics software and actuation hardware.

Finally, the work covers the dynamic characterization of the system. Through experimental tests and simulation in Simulink, the response of the motor-actuator assembly has been modeled, and a specific PID controller has been designed. Thanks to this, a final control architecture has been proposed where the main microcontroller handles the motor dynamics entirely, closing the speed loop using external sensor readings.

Contenido

Resum	2
Resumen	3
Abstract	4
Contenido	6
Glosario y Nomenclatura	9
Lista de figuras	11
Lista de tablas	13
1. Introducción	14
1.1. Motivación	14
1.2. Alcance del proyecto	14
1.3. Objetivos	15
2. Parte Teórica	17
2.1 Robot Operating System (ROS 2)	17
2.1.1 Introducción a ROS y evolución hacia ROS 2	17
2.1.2 Arquitectura de nodos, tópicos y mensajes	17
2.1.3 Modelo de comunicación publish/subscribe	19
2.1.4 Ventajas de ROS 2 en sistemas distribuidos y tiempo real	20
2.2 micro-ROS	20
2.2.1 Concepto de micro-ROS y motivación	20
2.2.2 Arquitectura de micro-ROS	21
2.2.3 Flujo de comunicación en micro-ROS	22
2.2.4 Limitaciones y consideraciones en microcontroladores	25
2.3 Comunicación serie UART / USART	26
2.3.1 Comunicación serie asíncrona	26
2.3.2 Protocolo UART y USART	26
2.3.3 Estructura, velocidad y sincronización	27
2.3.4 Uso de UART en sistemas embebidos y controladores de motor	28
2.4 Motores Brushless DC (BLDC)	29
2.4.1 Partes de un motor BLDC	30
2.4.2 Principio de funcionamiento de un motor BLDC	33
2.4.3 Control de motores BLDC	38
2.5 Controladores PID en sistemas dinámicos	40
2.5.1 Introducción a los controladores PID	40
2.5.2 Acción proporcional, integral y derivativa	41
2.5.3 Control PID aplicado a sistemas de primer orden	42
2.5.4 Aplicación del PID al control de velocidad de motores eléctricos	43
3. Metodología y equipos	44
3.1 Visión general del sistema desarrollado	44
3.2 Arquitectura del sistema distribuido	46
3.2.1 Justificación de la arquitectura distribuida	46

3.2.2 Nodos y responsabilidades del sistema	48
3.2.3 Flujo de datos extremo a extremo	50
3.3 Hardware utilizado	53
3.3.1 Microcontrolador principal (STM32 NUCLEO-F446ZE)	53
3.3.2 Controlador electrónico de velocidad (ESC B-G431B-ESC1)	55
3.3.3 Motor BLDC utilizado	57
Características generales del motor BLDC	58
3.3.4 Fuente de alimentación programable	60
3.3.5 Instrumentación de medida y depuración (osciloscopio)	61
3.4 Software utilizado	62
3.4.1 ROS 2 en el PC	62
3.4.2 micro-ROS en el sistema embebido	63
3.4.3 Entorno de desarrollo embebido (STM32CubeIDE y FreeRTOS)	63
3.4.4 Motor Control Workbench y configuración del ESC	64
3.4.5 Simulink y Simscape para simulación y diseño del control	65
3.5 Configuración del microcontrolador e integración de micro-ROS	65
3.5.1 Configuración de FreeRTOS	66
3.5.2 Configuración de la comunicación USART	67
3.5.3 Configuración de temporizadores y generación de PWM	68
3.5.4 Integración de micro-ROS en STM32CubeIDE	69
3.5.5 Validación de la integración	70
3.6 Implementación de un nodo ROS 2 para la publicación de la consigna de velocidad	70
3.6.1 Implementación del nodo publicador	71
3.6.2 Configuración para la ejecución del nodo	72
3.6.3 Papel del nodo en el sistema global	72
3.7 Transferencia de señal por UART	72
3.7.1 Arquitectura de comunicación serie	72
3.7.2 Suscripción al tópico y procesamiento de la consigna	73
3.7.3 Implementación de la transmisión UART	73
3.7.4 Configuración de los periféricos UART	75
3.7.5 Papel de la comunicación UART en el sistema	75
3.8 Recepción de la consigna de velocidad por UART y aplicación al motor	75
3.8.1 Estrategia de recepción y gestión del buffer	76
3.8.2 Procesamiento del mensaje y extracción de la velocidad	76
3.8.3 Aplicación de la consigna al control del motor BLDC	78
3.8.4 Integración dentro del sistema distribuido	79
4. Resultados y discusión	81
4.1 Validación del sistema de comunicación ROS 2 – micro-ROS	81
4.1.1 Ejecución del agente micro-ROS	81
4.1.2 Verificación del grafo de comunicación en ROS 2	82
4.1.3 Verificación de la publicación de la consigna de velocidad	83
4.2 Resultados de la transferencia de señal por UART	84
4.2.1 Transmisión de la consigna desde el STM32 por UART	85

4.2.2 Recepción e interpretación del comando por el controlador del motor	86
4.3 Simulación del sistema de control de velocidad del motor BLDC	87
4.3.1 Modelo utilizado en la simulación	88
4.3.2 Implementación del modelo en Simulink	89
4.3.3 Ajuste del controlador PID	90
4.3.4 Resultados de la simulación en lazo cerrado	92
4.3.5 Propuesta de implementación futura en el sistema físico	92
5. Planificación	95
6. Estudio económico	97
7. Estudio ambiental	101
8. Estudio social y de igualdad de género	103
9. Conclusiones	104
10. Agradecimientos	106
11. Bibliografía	107

Glosario y Nomenclatura

BLDC (Brushless DC): Motor de corriente continua sin escobillas.

DDS (Data Distribution Service): es el middleware estándar que permite la comunicación en tiempo real entre los diferentes componentes (nodos) de un robot.

ESC (Electronic Speed Controller): El variador o controlador de velocidad. Es la placa de potencia que se encarga de gestionar la electricidad para que el motor gire.

FOC (Field Oriented Control): Control vectorial. Una técnica de control avanzada para manejar el motor de forma muy precisa.

HAL (Hardware Abstraction Layer): Una capa de software que actúa como intermediario entre el sistema operativo y el hardware, permitiendo una comunicación uniforme.

IDE (Integrated Development Environment): El programa informático donde escribimos y depuramos el código (en este proyecto, STM32CubeIDE).

PID: El clásico controlador Proporcional-Integral-Derivativo. Es el algoritmo matemático que usamos para mantener la velocidad estable y corregir errores.

PWM (Pulse Width Modulation): Modulación por ancho de pulsos.

ROS 2 (Robot Operating System 2): es un marco de trabajo de código abierto y un conjunto de herramientas y bibliotecas para crear aplicaciones robóticas.

RTOS (Real-Time Operating System): Sistema operativo de tiempo real (aquí usamos FreeRTOS). Sirve para asegurar que las tareas importantes se ejecuten en un tiempo determinado.

UART (Universal Asynchronous Receiver-Transmitter): es un protocolo y hardware que permite la comunicación serial (bit a bit) entre dispositivos digitales.

XRCE-DDS: La versión "ligera" del protocolo DDS, optimizada para funcionar en microcontroladores con poca memoria.

Lista de figuras

Figura 2.1. Esquema de comunicación mediante tópicos y servicios en ROS 2. [19]

Figura 2.2. Arquitectura de pila de micro-ROS y su integración en el hardware. [22]

Figura 2.3. Esquema de conexión física UART (Tx-Rx). [27]

Figura 2.4. Partes de un Motor BLDC. [33]

Figura 2.5. Comparativa estructural: Motor DC con escobillas vs. Motor BLDC. [33]

Figura 2.6. Secuencia de conmutación de fases para el giro del motor BLDC. [35]

Figura 2.7. Diagrama de bloques control PID. [42]

Figura 3.1. Diagrama de bloques del sistema desarrollado. [45]

Figura 3.2. Diagrama de flujo de datos en la arquitectura distribuida del sistema. [50]

Figura 3.3. Placa de desarrollo NUCLEO-F446ZE utilizada. [53]

Figura 3.4. Controlador electrónico de velocidad (ESC) B-G431B-ESC1. [56]

Figura 3.5. Motor BLDC utilizado en las pruebas experimentales. [59]

Figura 3.6. Datos técnicos Maxon EC Motor 251601. [60]

Figura 3.7. Datos técnicos Maxon EC Motor 251601. [60]

Figura 3.8. Fuente de alimentación programable configurada para el sistema. [61]

Figura 3.9. Osciloscopio digital empleado para la verificación de señales. [62]

Figura 3.10. Selección del microcontrolador en STM32CubeMX. [66]

Figura 3.11. Configuración del middleware FreeRTOS. [67]

Figura 3.12. Parámetros de configuración del periférico USART2. [68]

Figura 3.13. Configuración del Timer 3 (TIM3) para generación de PWM. [69]

Figura 4.1. Ejecución del agente micro-ROS. [82]

Figura 4.2. Verificación tópico y suscripción al mismo. [83]

Figura 4.3. Publicación periódica de la consigna de velocidad. [84]

Figura 4.4. Transmisión UART en STM32CubeIDE. [86]

Figura 4.5. Montaje experimental completo con electrónica y motor conectados. [87]

Figura 4.6. Modelo de simulación de la planta en lazo abierto (Simulink). [88]

Figura 4.7. Respuesta temporal experimental para identificación del sistema. [89]

Figura 4.8. Diagrama de bloques del control PID de velocidad en lazo cerrado. [90]

Figura 4.9. Respuesta simulada del sistema con el controlador ajustado. [92]

Lista de tablas

Tabla 5.1. Planificación temporal del proyecto por fases y tareas [96]

Tabla 6.1. Estimación de la dedicación horaria y coste del trabajo personal [97]

Tabla 6.2. Desglose de costes operativos indirectos [98]

Tabla 6.3. Presupuesto de materiales, equipos y amortización [99]

Tabla 6.4. Coste total estimado del proyecto [100]

Tabla 7.1. Impacto Ambiental: Consumo Energético y Emisiones de CO2 [101]

1. Introducción

1.1. Motivación

Durante mi estancia de movilidad ERASMUS en Polonia surgió la oportunidad de orientar mi Trabajo de Fin de Grado hacia la robótica y los sistemas de control reales. En concreto, tras conversar con el coordinador de Control y Robótica de la Universidad Politécnica de Poznań, se me planteó la posibilidad de realizar una estancia académica centrada en el aprendizaje y uso del ecosistema ROS 2 y micro-ROS, con el objetivo de adquirir las bases necesarias para ser capaz de desarrollar un sistema de control para controlar una rueda robótica de reacción [1].

La propuesta consistía en familiarizarse progresivamente con herramientas software y hardware relacionadas con el control de motores y la robótica, abordando tanto la parte de robótica basada en ROS 2 como la integración con sistemas embebidos reales utilizando micro-ROS. Como caso de estudio, se planteaba el control de una rueda robótica, lo que implicaba trabajar con microcontroladores, electrónica de potencia y motores eléctricos, además de diseñar una arquitectura de comunicación distribuida.

Tras recibir esta propuesta, enseguida le confirmé a mi coordinador mi gran interés en realizar este ambicioso proyecto. Siempre me ha resultado especialmente motivador el ámbito de la robótica y la electrónica, y la posibilidad de aplicar conocimientos teóricos en un sistema físico real. Me pareció una oportunidad muy valiosa a nivel formativo. Además, el hecho de trabajar con tecnologías como ROS 2 desde cero supuso un reto personal importante, ya que no contaba con experiencia previa en este ecosistema.

La realización de este trabajo ha estado motivada por las ganas de aprender y consolidar conocimientos en sistemas de control distribuidos, comunicación entre dispositivos y control de motores, integrando software de alto nivel con hardware real. Este proceso de aprendizaje, iniciado durante la estancia en Polonia, ha sido uno de los principales motores del desarrollo del proyecto y ha marcado de forma clara el enfoque práctico y aplicado del Trabajo de Fin de Grado.

1.2. Alcance del proyecto

El alcance de este proyecto se centra en el diseño, implementación y validación de un sistema distribuido de control de velocidad para un motor BLDC, integrando un entorno basado en ROS 2 con un sistema embebido real y un controlador electrónico de potencia.

El trabajo abarca el desarrollo de un nodo ROS 2 ejecutado en un ordenador principal, encargado de generar y publicar consignas de velocidad, así como la integración de un microcontrolador STM32 dentro del ecosistema ROS 2 mediante micro-ROS. Este microcontrolador actúa como elemento intermedio, recibiendo las consignas de velocidad y transmitiéndolas al ESC a través de una comunicación serie UART.

Además, el proyecto incluye la configuración y utilización de un ESC capaz de gestionar internamente el control del motor BLDC, de manera que el sistema pueda tratar el conjunto motor-ESC como un actuador de velocidad. Este planteamiento permite abstraer la complejidad del control eléctrico y centrarse en el diseño del sistema distribuido y del control de alto nivel [17].

Adicionalmente, se realiza un estudio del comportamiento dinámico del sistema mediante simulación en Simulink, con el objetivo de modelar el conjunto motor-ESC de forma simplificada y diseñar un controlador PID de velocidad que permita analizar y justificar el comportamiento del sistema.

Quedan fuera del alcance de este trabajo la implementación directa de algoritmos de control de bajo nivel del motor, el diseño de la electrónica de potencia o la optimización del control interno del ESC, ya que estas tareas se delegan en herramientas y controladores especializados.

1.3. Objetivos

El objetivo general de este Trabajo de Fin de Grado es diseñar e implementar un sistema distribuido de control de velocidad para un motor BLDC, integrando un entorno ROS 2 de alto nivel con un sistema embebido real mediante micro-ROS, y validando su funcionamiento tanto a nivel de comunicación como de comportamiento dinámico.

De forma más concreta, los objetivos específicos del proyecto son los siguientes:

- Diseñar una arquitectura distribuida basada en ROS 2, micro-ROS y un microcontrolador STM32, que permita separar claramente los distintos niveles de control del sistema.
- Implementar un nodo ROS 2 capaz de generar y publicar consignas de velocidad de forma estructurada y modular.
- Integrar un nodo micro-ROS en el microcontrolador STM32 que permita recibir consignas desde ROS 2 y transmitir las al sistema de control del motor mediante UART.

- Configurar y utilizar un ESC comercial para el control interno de un motor BLDC, tratándolo como un actuador de velocidad dentro del sistema distribuido.
- Validar el correcto funcionamiento del sistema de comunicación ROS 2 – micro-ROS – UART mediante pruebas experimentales.
- Modelar el comportamiento dinámico del conjunto motor–ESC mediante simulación y diseñar un controlador PID de velocidad que permita analizar la respuesta del sistema.
- Demostrar la coherencia y viabilidad de la arquitectura propuesta, alineándola con enfoques utilizados en sistemas industriales y robóticos reales.

Con estos objetivos, el proyecto no solo persigue el control de un motor BLDC, sino también la adquisición de una visión integral sobre el diseño de sistemas distribuidos de control, combinando software de alto nivel, sistemas embebidos y electrónica de potencia de forma coherente y realista.

2. Parte Teórica

2.1 Robot Operating System (ROS 2)

2.1.1 Introducción a ROS y evolución hacia ROS 2

Robot Operating System (ROS) es un meta-sistema operativo de código abierto concebido específicamente para simplificar el desarrollo de aplicaciones robóticas [2]. Su propósito principal es ofrecer una infraestructura que facilite la integración de sensores, actuadores y algoritmos de control dentro de un mismo entorno. Más allá de la simple ejecución de programas, ROS pone a disposición del desarrollador un conjunto de herramientas orientadas a la comunicación entre procesos, la gestión del hardware y la organización modular del software, aspectos clave en sistemas robóticos complejos.

La primera versión de ROS se diseñó pensando, sobre todo, en entornos de investigación y prototipado. En ese contexto, la flexibilidad y la rapidez de desarrollo eran prioridades claras, incluso por delante de consideraciones como el tiempo real estricto o la robustez necesaria en aplicaciones industriales. Aunque esta versión tuvo una enorme aceptación, también dejó al descubierto ciertas limitaciones, especialmente cuando se aplicaba a sistemas distribuidos, aplicaciones críticas o escenarios con requisitos exigentes de latencia y fiabilidad [2].

ROS 2 aparece como una evolución natural de ROS, conservando sus ideas fundamentales pero replanteando su arquitectura para adaptarse mejor a las necesidades en la actualidad. Esta nueva versión aparece con la intención de mejorar la escalabilidad del sistema, ofrecer un soporte más sólido para arquitecturas distribuidas y facilitar la integración con aplicaciones en tiempo real. Además, ROS 2 amplía claramente su ámbito de aplicación, superando el entorno puramente académico y abriendo la puerta a su uso en sistemas industriales y plataformas embebidas.

2.1.2 Arquitectura de nodos, tópicos y mensajes

La arquitectura de ROS 2 se apoya en la idea de dividir el sistema en unidades de software independientes conocidas como nodos [2]. Cada nodo corresponde a un proceso que realiza una función específica, por ejemplo la lectura de un sensor, el cálculo de una consigna de control o el envío de órdenes a un actuador. Este enfoque evita el uso de aplicaciones monolíticas difíciles de mantener y, en su lugar, promueve una estructura modular que resulta más clara, flexible y fácil de ampliar con el tiempo.

La comunicación entre nodos no se realiza mediante llamadas directas a funciones, sino a través de un sistema de intercambio de mensajes. Estos mensajes siguen estructuras bien definidas, lo que permite desacoplar completamente los distintos componentes del sistema. Gracias a este planteamiento, un nodo puede modificarse, reemplazarse o ampliarse sin afectar al resto de la arquitectura, siempre que se mantenga la interfaz de comunicación establecida.

Los tópicos actúan como los canales por los que circula la información dentro del sistema. Cada tópico está asociado a un tipo de mensaje concreto, que define de manera precisa el formato de los datos transmitidos. Los nodos pueden publicar información en uno o varios tópicos y, del mismo modo, suscribirse a aquellos que contienen los datos necesarios para su funcionamiento. Este mecanismo hace posible que varios nodos compartan información de forma simultánea y eficiente, manteniendo una comunicación clara y organizada dentro del sistema de ROS 2 [2].

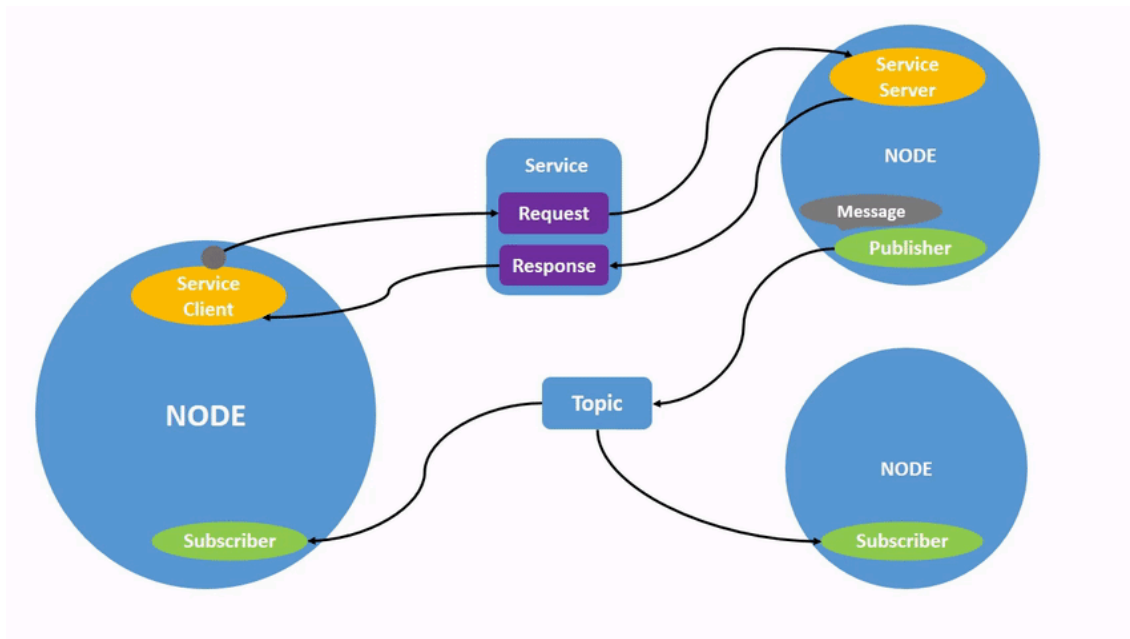


Figura 2.1. Esquema de comunicación mediante tópicos y servicios en ROS 2. Fuente: Documentación oficial de ROS 2 [2].

2.1.3 Modelo de comunicación *publish/subscribe*

El modelo de comunicación *publish/subscribe* es uno de los pilares sobre los que se apoya ROS 2. En este esquema, los nodos que actúan como publicadores envían información a un tópico concreto sin necesidad de saber qué otros nodos la recibirán. De manera equivalente, los nodos suscriptores acceden a los mensajes publicados en un tópico sin conocer qué proceso los ha generado. Esta separación entre emisor y receptor es clave para el funcionamiento del sistema [2].

Gracias a este enfoque, se consigue un elevado grado de desacoplamiento entre los distintos componentes. Los nodos no dependen directamente unos de otros, sino únicamente de los tópicos a los que están asociados. Esto aporta una gran flexibilidad al diseño del sistema y facilita su escalabilidad, ya que es posible añadir o eliminar nodos publicadores o suscriptores sin alterar el comportamiento del resto de la arquitectura.

Además, el modelo *publish/subscribe* encaja especialmente bien en sistemas distribuidos, donde los nodos pueden ejecutarse en dispositivos distintos conectados

mediante una red. En este escenario, ROS 2 se encarga de gestionar la comunicación de forma transparente para el desarrollador, de modo que el intercambio de información entre nodos se realiza de manera natural, como si todos ellos estuvieran ejecutándose en una misma máquina [2].

2.1.4 Ventajas de ROS 2 en sistemas distribuidos y tiempo real

Una de las diferencias más claras entre ROS 2 y su predecesor es su enfoque decidido hacia los sistemas distribuidos. La incorporación del estándar DDS (Data Distribution Service) como base del mecanismo de comunicación permite gestionar el intercambio de datos de forma más robusta y eficiente, incluso cuando los nodos se ejecutan en dispositivos distintos o dentro de redes con cierta complejidad [3].

Además, ROS 2 introduce el concepto de calidad de servicio (QoS), una característica clave que ofrece un control fino sobre cómo se transmiten los mensajes. A través de estas políticas es posible ajustar parámetros como la fiabilidad de la comunicación, la latencia, la prioridad de los datos o el comportamiento del sistema frente a pérdidas de información. Esto da al desarrollador la flexibilidad necesaria para adaptar el sistema a contextos muy diferentes, desde aplicaciones poco críticas hasta escenarios con requisitos estrictos de tiempo real [2].

Otro aspecto relevante es la mejora en la integración con sistemas embebidos y entornos en tiempo real. ROS 2 ha sido concebido desde el inicio para convivir con sistemas operativos en tiempo real y microcontroladores, lo que lo hace especialmente adecuado para arquitecturas híbridas en las que se combinan nodos de alto nivel con otros encargados del control de bajo nivel.

En conjunto, todas estas características convierten a ROS 2 en una plataforma idónea para el desarrollo de sistemas distribuidos complejos, como el realizado en este trabajo, donde distintos dispositivos cooperan mediante una comunicación estructurada con un mismo objetivo; el de gobernar un sistema físico real.

2.2 micro-ROS

2.2.1 Concepto de micro-ROS y motivación

Micro-ROS es una extensión del ecosistema ROS 2 creada para llevar sus capacidades a sistemas embebidos basados en microcontroladores con recursos limitados. Su propósito principal es permitir que elementos como sensores, actuadores o controladores de bajo nivel se integren de forma natural dentro de una arquitectura ROS 2, conservando la misma arquitectura de nodos, tópicos y mensajes que se utiliza en sistemas de mayor escala [4].

La motivación de fondo de micro-ROS es, por tanto, reducir la distancia tradicional entre el software de alto nivel y el hardware de bajo nivel. Gracias a esta extensión, los microcontroladores dejan de comportarse como simples periféricos aislados y pasan a formar parte activa del sistema distribuido, con la capacidad de publicar información y recibir consignas utilizando exactamente los mismos mecanismos que cualquier otro nodo ROS 2.

2.2.2 Arquitectura de micro-ROS

La arquitectura de micro-ROS conserva los principios fundamentales de ROS 2, pero los adapta cuidadosamente a las limitaciones propias de los sistemas embebidos. Al igual que en ROS 2, el sistema se estructura en nodos que pueden incluir publicadores, suscriptores, servicios y temporizadores. La diferencia está en cómo se implementan internamente estos elementos: todo está pensado para minimizar el uso de memoria y reducir la carga de procesamiento, algo clave cuando se trabaja con microcontroladores [4].

Para lograrlo, micro-ROS se apoya en el middleware Micro XRCE-DDS, una versión ligera del estándar DDS-XRCE (eXtremely Resource Constrained Environments). Este middleware ha sido diseñado específicamente para entornos con recursos muy limitados y permite que los microcontroladores se comuniquen con sistemas ROS 2 completos sin necesidad de ejecutar una pila DDS convencional en el propio dispositivo embebido. En la práctica, esto supone una enorme simplificación sin renunciar a la compatibilidad con el ecosistema ROS 2 [4,6].

Otra ventaja destacable de micro-ROS es su capacidad para adaptarse a distintos entornos de ejecución. Puede funcionar sobre sistemas operativos en tiempo real como FreeRTOS, Zephyr o NuttX, y también en configuraciones bare-metal, donde no existe un sistema operativo como tal. Esta versatilidad facilita su uso en una amplia

gama de plataformas de hardware, incluyendo microcontroladores como STM32, ESP32 o Raspberry Pi Pico [4].

Esta arquitectura es extremadamente útil, ya que permite programar aplicaciones embebidas siguiendo el mismo modelo conceptual que en ROS 2. Esto simplifica notablemente la integración con el resto del sistema y reduce el esfuerzo de mantenimiento a medida que el proyecto crece.

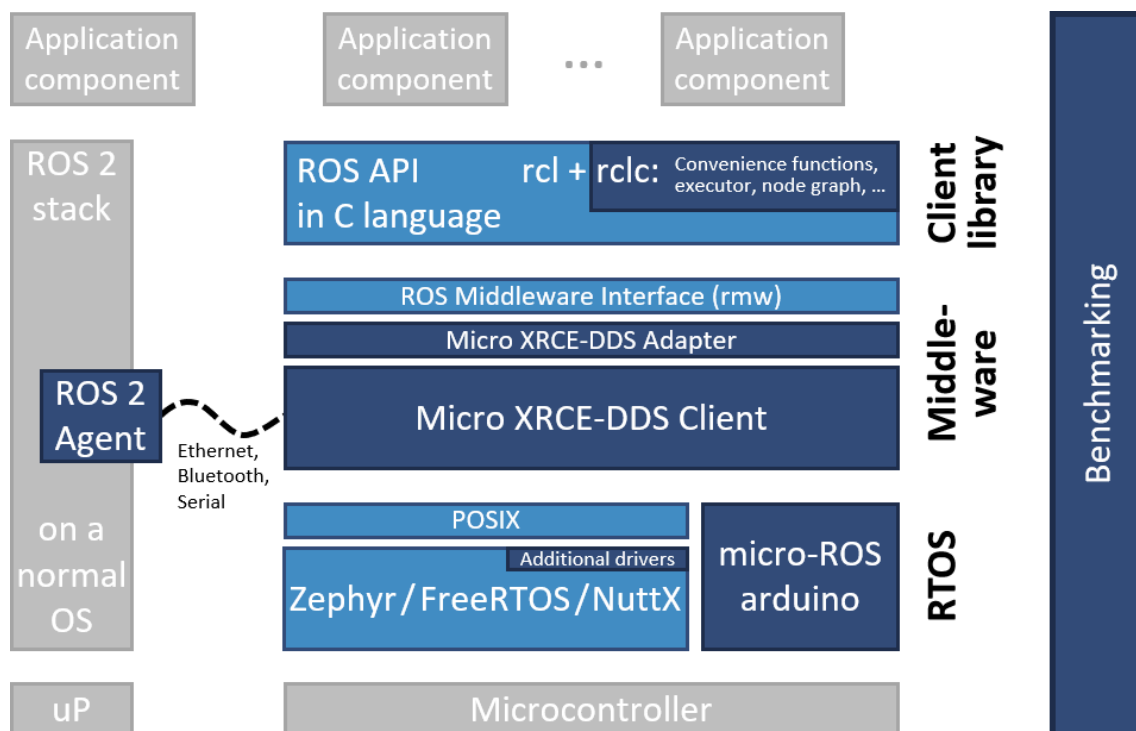


Figura 2.2. Arquitectura de pila de micro-ROS y su integración en el hardware. Fuente: micro.ros.org.

2.2.3 Flujo de comunicación en micro-ROS

El flujo de comunicación en micro-ROS resulta relativamente similar al de ROS 2, es decir, se basa en el intercambio de mensajes entre nodos mediante el modelo publish/subscribe. Cada nodo puede contar con uno o varios publicadores, encargados de enviar información, así como con suscriptores que reciben los datos asociados a los tópicos de su interés. Esta forma de comunicarse resulta bastante natural y coherente con el resto del ecosistema ROS, incluso cuando se trabaja en entornos embebidos.

Los mensajes viajan a través de tópicos, que funcionan como canales de comunicación unidireccionales y están ligados a un tipo de dato concreto. Gracias a esta estructura, es posible construir sistemas muy modulares, donde cada nodo se centra en una tarea específica sin necesidad de conocer el funcionamiento interno del resto. En la práctica, esto implica sistemas más flexibles, escalables y sencillos de mantener, ya que añadir o modificar nodos no obliga a rehacer toda la arquitectura [4].

Ahora bien, existe una diferencia clave respecto a los nodos ROS 2 convencionales. Un nodo micro-ROS no se conecta directamente a la red DDS. En su lugar, toda la comunicación pasa por un componente intermedio conocido como agente micro-ROS. Este agente actúa como un puente entre el microcontrolador y el resto del sistema ROS 2 y se encarga de traducir y reenviar los mensajes. De este modo, el microcontrolador puede integrarse plenamente en el sistema distribuido sin necesidad de ejecutar una pila DDS completa, algo que sería inviable teniendo en cuenta sus limitaciones de recursos [4].

Agente micro-ROS

El agente micro-ROS es un proceso que se ejecuta en un equipo con mayor capacidad de cómputo, normalmente un ordenador personal o una placa con un sistema operativo completo. Su papel es clave dentro de la arquitectura, ya que se encarga de recibir los mensajes procedentes del microcontrolador, adaptarlos al formato utilizado por ROS 2 y reenviarlos al resto de nodos del sistema. Este proceso también funciona en sentido inverso, permitiendo que los mensajes generados en ROS 2 lleguen correctamente al entorno embebido [4].

En la práctica, el agente asume toda la complejidad asociada a la comunicación DDS, descargando al microcontrolador de esta tarea. Desde el punto de vista del sistema ROS 2, el agente se comporta como un nodo más dentro del grafo de comunicación. Para el microcontrolador, en cambio, el agente representa el único punto de entrada y salida hacia el ecosistema ROS 2, simplificando de forma considerable la gestión de las comunicaciones.

Gracias a esta separación de responsabilidades, el microcontrolador puede integrarse de forma natural en un sistema distribuido sin la necesidad de implementar una pila de comunicación compleja. Esto es importante cuando se trabaja con dispositivos con

recursos limitados, donde la memoria y la capacidad de procesamiento son factores críticos y relevantes.

Cliente micro-ROS

El cliente micro-ROS se ejecuta directamente en el microcontrolador y es el encargado de crear y gestionar los nodos embebidos que forman parte de la aplicación. A través de este cliente se definen los publicadores y suscriptores necesarios, permitiendo de esta manera que el microcontrolador intercambie información con el resto del sistema. Para ello, el cliente establece una conexión con el agente micro-ROS mediante un medio de transporte adecuado, como UART, USB o Ethernet, en función de las capacidades de la plataforma utilizada [4].

Trabajar con el cliente micro-ROS resulta bastante natural, ya que la interfaz y los conceptos utilizados son muy similares a los de ROS 2. Esto facilita la reutilización de conocimientos previos y reduce de forma notable la dificultad de pasar del entorno de alto nivel al embebido. No obstante, el cliente está optimizado para adaptarse a las restricciones del hardware, minimizando el uso de memoria dinámica y ajustando el consumo de recursos a las posibilidades reales del microcontrolador.

Serialización de mensajes

Un elemento especialmente importante dentro del flujo de comunicación de micro-ROS es la serialización de los mensajes. Antes de que los datos generados en el microcontrolador puedan enviarse, es necesario transformarlos en un formato binario compacto, pensado para viajar de forma eficiente a través del canal de comunicación disponible.

Este proceso no es un simple detalle técnico: la serialización permite reducir de manera significativa el tamaño de los mensajes y optimizar la transmisión, algo realmente importante cuando se trabaja con enlaces de ancho de banda limitado o con microcontroladores que cuentan con recursos ajustados. Una vez estos datos llegan al agente micro-ROS, el proceso se invierte. Los mensajes se deserializan y se convierten de nuevo en mensajes ROS 2 estándar, de modo que el resto de nodos del

sistema pueden tratarlos como si provinieran de cualquier otro nodo ROS 2 convencional, sin necesidad de preocuparse por su origen embebido [2,4].

2.2.4 Limitaciones y consideraciones en microcontroladores

A pesar de todas las ventajas que aporta, el uso de micro-ROS en microcontroladores también implica una serie de limitaciones que conviene tener muy presentes durante el diseño del sistema. La más evidente tiene que ver con los recursos disponibles. La memoria, tanto RAM como de programa, suele ser reducida, lo que obliga a ser cuidadoso con el número de nodos, tópicos y mensajes que se definen, además de diseñar estructuras de datos lo más compactas posible.

Otro aspecto a tener en cuenta es el tiempo de ejecución. Aunque micro-ROS está pensado para integrarse en sistemas en tiempo real, la comunicación con el agente introduce una cierta latencia adicional. Este retardo no suele resultar un problema en tareas de supervisión o en el envío de consignas, pero sí debe considerarse en aplicaciones especialmente críticas. Por este motivo, lo más habitual es reservar micro-ROS para el control de alto nivel o la coordinación del sistema, mientras que los lazos de control más rápidos se implementan de forma local en el propio microcontrolador o directamente en hardware dedicado.

Por último, la dependencia del agente micro-ROS es otro factor a tener en cuenta. Para que la comunicación con el resto del ecosistema ROS 2 funcione correctamente, el agente debe estar en ejecución. En aplicaciones donde se requiera un funcionamiento completamente autónomo del sistema embebido, este aspecto debe de considerarse de cara al diseño del sistema y de sus objetivos específicos.

Pese a estas restricciones, micro-ROS se presenta como una solución muy eficaz para integrar microcontroladores dentro de arquitecturas ROS 2 distribuidas. Permite combinar la flexibilidad y potencia del entorno ROS con el control directo del hardware embebido, de forma que se logra un equilibrio muy interesante entre capacidad y eficiencia.

2.3 Comunicación serie UART / USART

2.3.1 Comunicación serie asíncrona

La comunicación serie asíncrona es uno de los métodos más habituales para el intercambio de datos entre dispositivos electrónicos, especialmente dentro del ámbito de los sistemas embebidos. A diferencia de la comunicación paralela, donde varios bits se transmiten al mismo tiempo, en este caso la información se envía bit a bit a través de una o muy pocas líneas de comunicación. Esto permite reducir considerablemente el número de conexiones físicas necesarias y, por tanto, simplificar tanto el diseño como el cableado del hardware.

En este tipo de comunicación no existe una señal de reloj compartida entre el emisor y el receptor. En su lugar, ambos dispositivos deben acordar previamente una velocidad de transmisión, de forma que el receptor sea capaz de interpretar correctamente los bits recibidos basándose únicamente en el tiempo. La verdad es que esta característica aporta una gran flexibilidad, ya que permite comunicar dispositivos que no comparten el mismo reloj interno o que incluso se encuentran físicamente separados.

Gracias a su sencillez y versatilidad, la comunicación serie asíncrona se ha convertido en una solución muy extendida para tareas como la depuración de sistemas, la configuración de dispositivos o la comunicación entre microcontroladores y periféricos externos. No obstante, esta simplicidad también conlleva algunas limitaciones, como una velocidad de transmisión inferior a la de otros protocolos síncronos y una mayor sensibilidad a posibles errores de temporización si los parámetros no están bien ajustados.

2.3.2 Protocolo UART y USART

UART (Universal Asynchronous Receiver/Transmitter) es, sin duda, uno de los protocolos más utilizados para implementar comunicación serie asíncrona en sistemas embebidos. Se basa en una comunicación punto a punto y, en su forma más básica, requiere únicamente dos líneas: una dedicada a la transmisión de datos (TX) y otra a la recepción (RX). En función de la aplicación, pueden añadirse líneas adicionales para control de flujo, aunque en muchos casos no son estrictamente necesarias.

USART (Universal Synchronous/Asynchronous Receiver/Transmitter) puede entenderse como una evolución del UART clásico. Este periférico permite trabajar tanto en modo asíncrono como en modo síncrono. En el modo síncrono se incorpora una señal de reloj compartida entre emisor y receptor, lo que mejora la sincronización y permite alcanzar velocidades de transmisión más elevadas. Aun así, la realidad es que en un gran número de aplicaciones prácticas se opta por el modo asíncrono, ya que resulta más sencillo de implementar y requiere menos conexiones físicas.

En microcontroladores STM32, los módulos USART están integrados directamente como periféricos hardware. Esto permite gestionar la transmisión y recepción de datos de forma eficiente, descargando al procesador de gran parte del trabajo. Gracias a esta integración, la comunicación serie puede coexistir sin problemas con otras tareas del sistema, lo que resulta especialmente importante en aplicaciones en tiempo real o con recursos limitados [16].

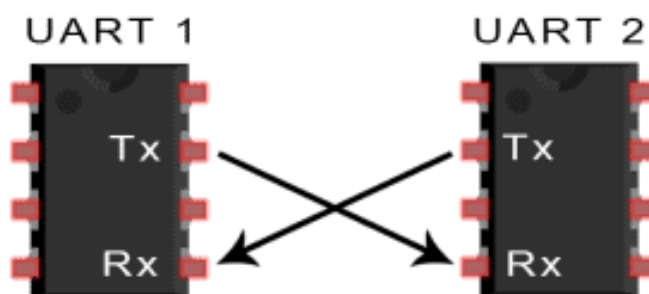


Figura 2.3. Esquema de conexión física UART (Tx-Rx). Fuente: MCI Electronics

2.3.3 Estructura, velocidad y sincronización

En la comunicación UART, la información no se envía de forma continua, sino organizada en tramas bien definidas. Cada una de estas tramas comienza con un bit de inicio, cuya función es avisar al receptor de que va a empezar una transmisión. A partir de ahí se envían los bits de datos propiamente dichos, cuyo número depende de la configuración elegida. Para cerrar la trama, se añaden uno o varios bits de parada, que indican claramente que el envío ha finalizado.

De manera opcional, puede incluirse también un bit de paridad. Este bit aporta un mecanismo sencillo de detección de errores, ya que permite comprobar si el número de bits transmitidos cumple una determinada condición. Eso sí, conviene tener en

cuenta que se trata de una verificación básica y que no ofrece capacidades avanzadas de corrección de errores.

La velocidad de transmisión, conocida habitualmente como “baud rate”, establece cuántos bits se envían por segundo. Para que la comunicación funcione correctamente, emisor y receptor deben estar configurados exactamente con el mismo valor. Incluso pequeñas diferencias pueden provocar desajustes temporales y, en consecuencia, errores en la interpretación de los datos.

Para garantizar una comunicación robusta, es fundamental que la duración de los impulsos sea lo más precisa posible. El tiempo asignado a cada bit (T_{bit}), que define cuánto debe mantenerse estable un nivel lógico antes del siguiente, se calcula simplemente como el inverso de la velocidad de transmisión o “baud rate” configurada.

$$T_{\text{bit}} = \frac{1}{\text{BaudRate}}$$

A partir de este valor, se puede determinar el tiempo total que tarda en enviarse una trama completa (T_{frame}), teniendo en cuenta no solo los datos sino también el conjunto de bits de protocolo (inicio, parada y paridad):

$$T_{\text{frame}} = \frac{N_{\text{bitstotales}}}{\text{BaudRate}}$$

Ajustar estos intervalos con exactitud es extremadamente importante ya que permite al receptor realizar el muestreo justo en la mitad de cada bit. De esta forma, se asegura que la lectura sea correcta incluso si aparecen pequeñas desincronizaciones entre los relojes de ambos dispositivos conectados.

2.3.4 Uso de UART en sistemas embebidos y controladores de motor

En el ámbito de los sistemas embebidos, UART se ha consolidado como uno de los medios de comunicación más habituales entre microcontroladores y dispositivos externos, como sensores, módulos de comunicación o controladores de motor. Su bajo coste de implementación, junto con la amplia compatibilidad con todo tipo de hardware, hace que sea una opción recurrente tanto en aplicaciones industriales como en entornos de investigación y prototipado [21].

En el caso concreto de los sistemas de control de motores, la comunicación UART permite enviar consignas de control —por ejemplo, valores de velocidad o par— y, al mismo tiempo, recibir información relevante sobre el estado del sistema, como la velocidad medida, la presencia de errores o determinadas condiciones de funcionamiento. Este enfoque resulta especialmente práctico cuando el control de bajo nivel del motor está encapsulado en un módulo específico, como un ESC, y el microcontrolador se encarga principalmente de tareas de supervisión o de control de alto nivel [21].

Además, UART suele desempeñar un papel clave durante la fase de desarrollo y puesta a punto del sistema. Es habitual utilizar esta interfaz como canal de depuración y monitorización, ya que permite observar el comportamiento interno del controlador y ajustar parámetros de forma rápida, sin necesidad de modificar el hardware ni añadir instrumentación adicional.

En este trabajo, la comunicación UART se emplea precisamente como enlace entre el microcontrolador y el controlador electrónico de velocidad. Este canal constituye un elemento central en la integración del sistema distribuido, ya que gracias a él es posible transmitir de manera sencilla y fiable las consignas generadas en el entorno ROS 2 hasta el sistema encargado del control físico del motor.

2.4 Motores Brushless DC (BLDC)

A diferencia de los motores de corriente continua con escobillas, en los BLDC la conmutación de la corriente no se realiza a través de elementos mecánicos, sino de forma electrónica a través de un controlador externo, por lo tanto, elimina el desgaste asociado a las escobillas y mejora notablemente la fiabilidad del conjunto.

Desde un punto de vista funcional, un motor BLDC se puede interpretar como una máquina síncrona de imanes permanentes. El rotor gira solidariamente con el campo magnético generado en el estator, de modo que ambos permanecen sincronizados en todo momento. Esta propiedad resulta especialmente interesante, ya que permite regular con precisión variables como la velocidad o el par, siempre que la excitación de las fases del estator se realice de forma adecuada y esté correctamente alineada con la posición del rotor [7,8].

Gracias a estas características, los motores BLDC encajan especialmente bien en sistemas donde se busca un control del movimiento eficiente, fiable y con buenas prestaciones dinámicas.

2.4.1 Partes de un motor BLDC

Un motor BLDC trifásico se compone de dos elementos clave: el estator y el rotor. Es precisamente gracias a la interacción entre ellos por lo que se da lugar a la aparición del par electromagnético y al movimiento del eje del motor.

Estator trifásico

El estator es la parte fija del motor y está constituido por un conjunto de bobinados distribuidos de forma regular a lo largo de su perímetro. En la mayoría de motores BLDC, estos bobinados se organizan en tres fases independientes, separadas entre sí 120 grados eléctricos. Cada una de estas fases está formada por uno o varios devanados que, al ser recorridos por corriente eléctrica, generan un campo magnético [7,8].

Gracias a esta disposición trifásica, es posible crear un campo magnético giratorio cuando las corrientes aplicadas a las distintas fases siguen una secuencia adecuada. Este campo giratorio es el elemento central del funcionamiento del motor, ya que interactúa directamente con el campo magnético del rotor y provoca la aparición del par que hace girar el eje [7,8].

Desde el punto de vista del control, el estator es el punto de actuación directa de la electrónica de potencia. La amplitud, la forma de onda y el orden en que se excitan sus fases condicionan de forma directa el comportamiento del motor. La velocidad alcanzada, el par desarrollado o incluso la suavidad del giro dependen de cómo se gestionen estas corrientes, lo que explica la importancia del controlador electrónico en los motores BLDC [7].

Rotor con imanes permanentes

El rotor constituye la parte móvil del motor y está formado por uno o varios imanes permanentes dispuestos de manera que generan un campo magnético fijo. Según el diseño del motor, estos imanes se puede colocar sobre la superficie del rotor o integrarse en su interior, una decisión que influye tanto en las prestaciones como en el comportamiento mecánico.

El campo magnético generado por el rotor interactúa directamente con el campo creado en el estator. Cuando, mediante la conmutación electrónica adecuada, el campo del estator comienza a girar, el rotor tiende de forma natural a alinearse con él. Es este intento constante de alineación el que da lugar al movimiento de rotación del eje del motor [8].

Una de las principales ventajas del uso de imanes permanentes es que el rotor no necesita ser alimentado eléctricamente. Esto reduce de manera notable las pérdidas asociadas y contribuye a una mayor eficiencia global del motor. Por otro lado, esta característica también impone ciertas exigencias al sistema de control, que debe tener en cuenta las propiedades magnéticas del rotor. En particular, aspectos como la correcta sincronización entre rotor y estator, así como los límites de velocidad admisibles, pasan a ser factores relevantes para garantizar un funcionamiento estable y seguro del motor.

Comparación con motores DC con escobillas

Una de las diferencias más claras entre los motores BLDC y los motores de corriente continua con escobillas está en la forma en que se realiza la conmutación. En los motores DC convencionales, su funcionamiento está basado en un sistema mecánico formado por escobillas y un colector, que se encargan de invertir automáticamente la corriente en el rotor. Esta solución simplifica el control, pero tiene un precio: desgaste mecánico, generación de chispas, mayor mantenimiento y pérdidas energéticas por culpa del rozamiento.

En los motores BLDC, por el contrario, la conmutación se lleva a cabo de manera totalmente electrónica. Es el controlador el que decide en cada instante qué fases deben alimentarse y con qué intensidad, en función de la posición del rotor. Esta diferencia fundamental permite eliminar los problemas derivados del contacto físico, alargando la vida útil del motor y mejorando su eficiencia global [8].

Eso sí, esta ventaja no es gratuita. Al prescindir de la conmutación mecánica, el motor BLDC necesita necesariamente un sistema de control capaz de gestionar correctamente la excitación de las fases. Sin ese controlador electrónico, el motor simplemente no puede funcionar, lo que introduce una mayor complejidad en el conjunto.

Ventajas e inconvenientes de los motores BLDC

Entre las principales ventajas de los motores BLDC destaca, en primer lugar, su elevada eficiencia y su alta densidad de potencia. La ausencia de escobillas elimina una de las fuentes más habituales de fallo mecánico, algo muy importante en aplicaciones donde se exige un funcionamiento continuo y fiable.

Además, estos motores ofrecen un control muy preciso de variables como la velocidad o el par. Su buena respuesta dinámica y la posibilidad de trabajar a altas velocidades hacen que sean muy atractivos para sistemas de control avanzados.

Como principal inconveniente, los motores BLDC requieren un sistema de control más sofisticado que otros tipos de motores. La conmutación electrónica obliga a emplear electrónica de potencia, sensores de posición o algoritmos de estimación, además de un mayor esfuerzo de diseño. Dicho esto, los avances recientes en microcontroladores y controladores de motor han reducido notablemente esta complejidad, haciendo que los motores BLDC sean hoy en día una opción realmente accesible.

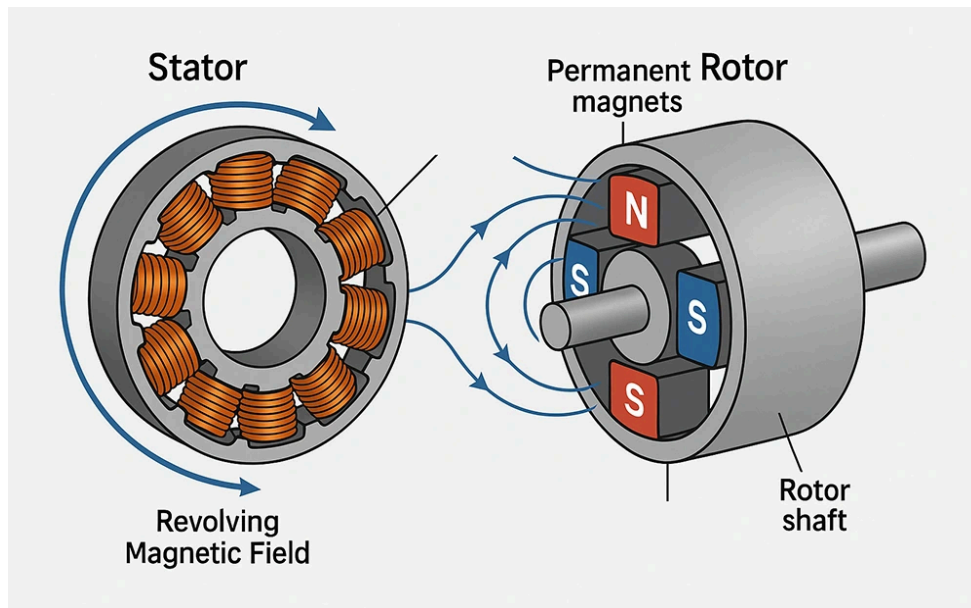


Figura 2.4. Partes de un Motor BLDC. Fuente: Gator Lamination

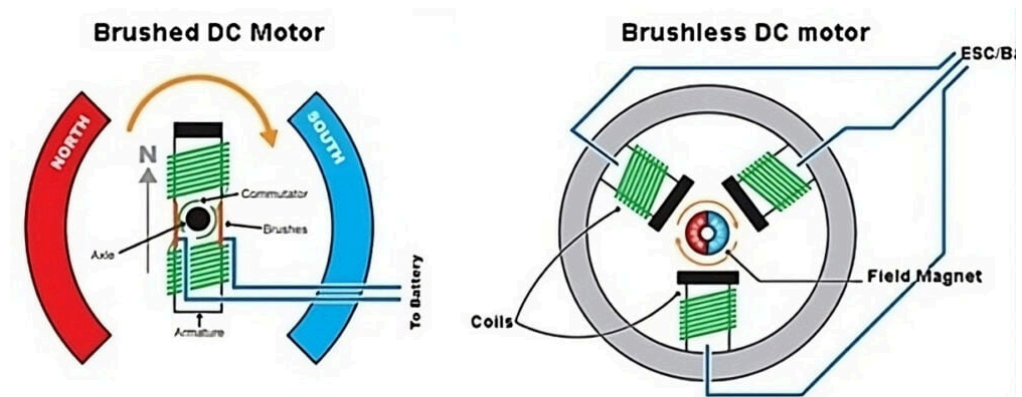


Figura 2.5. Comparativa estructural: Motor DC con escobillas vs. Motor BLDC. Fuente: Renesas Electronics.

2.4.2 Principio de funcionamiento de un motor BLDC

El funcionamiento de un motor BLDC se basa en una idea clave: generar un campo magnético giratorio en el estator y hacer que este interactúe de forma controlada con el campo magnético creado por los imanes permanentes del rotor. A diferencia de los motores de corriente continua con escobillas, donde la conmutación se produce de manera mecánica casi “automática”, en un motor BLDC todo el movimiento depende de que la excitación de las fases del estator esté perfectamente sincronizada con la

posición del rotor. Si esa sincronización falla, el motor simplemente no gira de forma correcta [7,9].

Campo magnético giratorio

El campo magnético giratorio en un motor BLDC se obtiene alimentando de manera secuencial los devanados del estator. En el caso más habitual, el de un motor trifásico, las tres fases se encuentran distribuidas espacialmente con un desfase de 120 grados eléctricos entre ellas. Cuando se aplican corrientes a estas fases siguiendo una secuencia adecuada, el efecto combinado de sus campos magnéticos da lugar a un campo resultante que parece girar alrededor del eje del motor [7,8,9].

Conviene destacar que este campo giratorio no es un fenómeno independiente en sí mismo, sino el resultado de la superposición de los campos generados por cada fase del estator. La velocidad a la que gira este campo viene determinada por la frecuencia con la que se conmuta la excitación de las fases, mientras que su intensidad está directamente relacionada con la magnitud de la corriente aplicada [7,9].

La correcta generación de este campo magnético giratorio es fundamental para el funcionamiento del motor. De ella dependen aspectos tan importantes como la dirección de giro, la estabilidad del movimiento y la capacidad del motor para desarrollar par de forma continua y controlada.

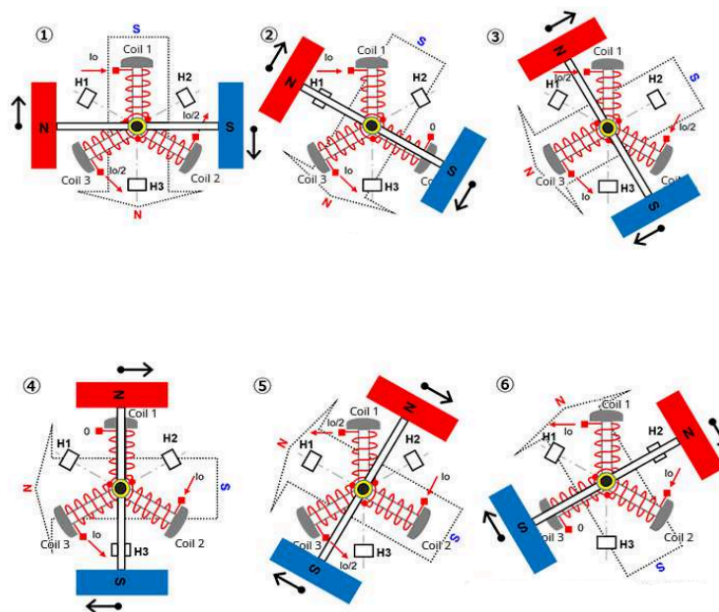


Figura 2.6. Secuencia de conmutación de fases para el giro del motor BLDC.

Fuente: Nanotec.

Interacción rotor–estator

El rotor de un motor BLDC incorpora imanes permanentes que generan un campo magnético fijo. Cuando este campo interactúa con el campo magnético giratorio creado en el estator, aparece una fuerza que tiende a alinear dichos campos. Es precisamente este fenómeno el que da lugar al par electromagnético y, por tanto, al movimiento del rotor.

Para que el giro sea continuo y estable, el campo del estator no puede quedarse simplemente alineado con el del rotor. Al contrario, debe adelantarse de forma progresiva. Si ambos campos permanecieran estáticos y perfectamente alineados, el motor no produciría movimiento alguno. Por este motivo, la electrónica de control es tan relevante: debe ajustar de manera constante la excitación de las fases del estator en función de la posición instantánea del rotor, asegurando que el campo magnético giratorio “tire” del rotor y lo haga avanzar de forma controlada [7,8].

Esta relación tan estrecha entre la velocidad del rotor y la del campo magnético del estator es lo que permite clasificar al motor BLDC como una máquina síncrona. En condiciones normales de funcionamiento y siempre que no se superen los límites de par del sistema, el rotor gira siguiendo exactamente la velocidad del campo magnético generado en el estator.

Generación del par electromagnético

El par electromagnético en un motor BLDC aparece como consecuencia directa de la interacción entre el campo magnético generado por el rotor y el campo creado en el estator. Desde un punto de vista físico, esta interacción es más efectiva cuando el ángulo entre los campos se sitúa alrededor de los 90 grados eléctricos. En esa situación, la capacidad del motor para generar par es máxima [8].

Uno de los objetivos principales de la electrónica de control es mantener esta condición de funcionamiento de forma aproximada a lo largo del tiempo. Para ello, se ajustan de manera continua tanto la secuencia como la amplitud de las corrientes que circulan por las fases del estator. De esta forma, se logra una generación de par estable, evitando pérdidas innecesarias y reduciendo posibles oscilaciones que podrían afectar al comportamiento del motor.

La cantidad de par desarrollada por el motor está muy relacionada con la intensidad de la corriente que atraviesa los devanados del estator. Por tanto, en los motores BLDC el control del par no suele realizarse de forma directa, sino de manera indirecta a través del control de la corriente. Este enfoque puede implementarse mediante estrategias relativamente simples de modulación o mediante técnicas más avanzadas, como el control vectorial, que permiten un ajuste más preciso y suave del par generado [7,8].

Relación corriente-par-velocidad

Dentro del rango de funcionamiento lineal del motor, la relación entre la corriente que se aplica a las fases del estator y el par electromagnético generado es, de forma aproximada, proporcional. Esta relación queda definida por la constante de par del motor, un parámetro que vincula directamente la intensidad de corriente con la capacidad del motor para producir par.

La velocidad de giro, sin embargo, no depende únicamente de la corriente aplicada. En realidad, viene determinada por el equilibrio entre el par electromagnético generado y las pérdidas mecánicas del sistema, como el rozamiento interno o la carga conectada al eje. Si el par desarrollado supera estas pérdidas, el motor acelera;

cuando ambos efectos se compensan, se alcanza una velocidad constante en régimen permanente [8,9].

Esta interacción entre corriente, par y velocidad resulta clave a la hora de diseñar sistemas de control. En la práctica, el controlador actúa regulando la corriente para generar el par necesario que permita alcanzar y mantener la velocidad deseada, incluso en presencia de perturbaciones externas o cambios en la carga. Este principio constituye la base tanto de los esquemas de control de velocidad más sencillos como de las estrategias avanzadas utilizadas en motores BLDC [7,9].

Modelado Matemático del Motor BLDC

Para un análisis mas profundo del comportamiento del motor BLDC, es necesario establecer el modelo matemático que rige el comportamiento dinámico del motor. La dinámica eléctrica de cada fase se define mediante la ecuación de equilibrio de tensiones:

$$V = R \cdot i + L \frac{di}{dt} + e \quad \text{Ecuación (2.1)}$$

Donde V representa la tensión aplicada, R la resistencia del devanado, L su inductancia y e la fuerza contraelectromotriz (BEMF). Esta última se relaciona de forma lineal con la velocidad angular del motor (w) a través de la constante de fuerza contraelectromotriz (K_e):

$$e = K_e \cdot w \quad \text{Ecuación (2.2)}$$

El par electromagnético generado (T_e) es proporcional a la corriente de fase (i) a través de la constante de par (K_t):

$$T_e = K_t \cdot i \quad \text{Ecuación (2.3)}$$

Finalmente, la dinámica mecánica del conjunto se describe aplicando la segunda ley de Newton al eje del motor:

$$J \frac{dw}{dt} = T_e - T_L - B \cdot w \quad \text{Ecuación (2.4)}$$

Siendo J el momento de inercia, T_L el par de carga externo y B el coeficiente de fricción viscosa [28].

2.4.3 Control de motores BLDC

El control de un motor BLDC es más complejo que el de un motor de corriente continua tradicional, ya que requiere sincronizar la activación de las bobinas del estator con la posición del rotor. A diferencia de los motores con escobillas, donde la conmutación es mecánica, aquí es la electrónica la que debe decidir cuándo cambiar la corriente de fase para mantener el par constante.

Existen principalmente dos grandes estrategias para gestionar esta conmutación, dependiendo de la forma de onda de la corriente y la complejidad del algoritmo: la conmutación trapezoidal (bloque) y el control vectorial (FOC) [7,29,12].

Conmutación Trapezoidal (Control de 6 pasos)

Es el método más básico y sencillo de implementar. Se basa en alimentar dos fases del motor en cada instante, dejando la tercera flotante. Esto divide una vuelta eléctrica en seis pasos o sectores de 60 grados.

En modo sensorless, el instante de conmutación se determina mediante la detección del cruce por cero (Zero Crossing) de la fuerza contraelectromotriz (BEMF) en la fase no alimentada. La tensión inducida en esta fase responde a la expresión:

$$e(t) = K_e \cdot w_m \cdot f(\theta) \quad \text{Ecuación (2.5)}$$

Donde K_e es la constante de fuerza contraelectromotriz y w_m la velocidad angular del rotor. El controlador detecta un punto en el que $e(t) = 0$ y tras un retardo de 30 grados eléctricos estimado mediante el tiempo entre pasos previos, ejecuta el cambio de fase. Esta dependencia de la velocidad hace que el par generado presente un rizado (torque ripple) característico debido a las variaciones bruscas de corriente.

- Funcionamiento: El controlador detecta la posición del rotor (usualmente con sensores de efecto Hall) y conmuta los transistores del puente inversor bruscamente cada 60 grados [7].
- Ventajas: Es robusto y computacionalmente muy ligero.
- Inconvenientes: Genera un par con rizado (torque ripple), lo que provoca vibraciones y ruido, especialmente a bajas velocidades. No es ideal para aplicaciones de robótica de precisión.

Control Orientado a Campo (FOC)

El Control Orientado a Campo (Field Oriented Control) es una técnica mucho más avanzada que busca controlar el motor como si fuera de corriente continua,

desacoplando el par y el flujo magnético. En lugar de conmutar bruscamente, el FOC inyecta corrientes sinusoidales en las tres fases simultáneamente [12].

Para poder controlar estas corrientes de forma precisa, el microcontrolador necesita transformar las señales trifásicas en valores que se puedan procesar fácilmente. Este proceso matemático se divide en dos pasos:

Transformada de Clarke: El motor tiene tres corrientes físicas (i_a, i_b, i_c) medidas a partir de los shunts de la placa. La transformada de Clarke permite simplificar estas tres variables en un sistema de dos ejes (α y β) que están fijas en el estator. Físicamente se trata de descomponer los vectores de corriente de cada bobina mediante proyecciones trigonométricas y aplicar un factor de escala para mantener la amplitud de las señales [29]:

$$i_\alpha = i_a \cdot \cos(0) + i_b \cdot \cos(120) + i_c \cdot \cos(240) \quad \text{Ecuación (2.6)}$$

$$i_\beta = i_a \cdot \sin(0) + i_b \cdot \sin(120) + i_c \cdot \sin(240) \quad \text{Ecuación (2.7)}$$

$$\begin{pmatrix} i_\alpha \\ i_\beta \end{pmatrix} = \frac{2}{3} \begin{pmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{pmatrix} \begin{pmatrix} i_a \\ i_b \\ i_c \end{pmatrix} \quad \text{Ecuación (2.8)}$$

Transformada de Park : Aunque solo se trabaja con dos ejes, las señales siguen siendo alternas. Para controlarlas como si fueran corriente continua, se aplica la transformada de Park. La idea es "rotar" el sistema de referencia para que gire a la misma velocidad que el rotor, usando el ángulo medido por los sensores (θ). Al hacer esto, las corrientes pasan a ser valores constantes [29]:

$$\begin{pmatrix} i_d \\ i_q \end{pmatrix} = \begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix} \begin{pmatrix} i_\alpha \\ i_\beta \end{pmatrix} \quad \text{Ecuación (2.9)}$$

- i_q (Componente de cuadratura): Genera par motor. Es la variable que se trata de maximizar.

- I_d (Componente directa): Genera flujo magnético que no contribuye al giro y solo sobrecalienta el motor. Por lo general, el algoritmo trabaja para mantener esta componente en cero ($I_d = 0$).

Este método permite un movimiento suave, silencioso y eficiente.

Métodos de detección de posición

Para aplicar cualquiera de estos controles, es vital saber dónde está el rotor. Existen dos enfoques:

- Con Sensores (Sensored): Se utilizan dispositivos físicos como sensores Hall o encoders acoplados al eje. Ofrecen precisión total incluso con el motor parado, pero añaden coste, cableado y posibles puntos de fallo.
- Sin Sensores (Sensorless): El controlador estima la posición midiendo la fuerza contraelectromotriz (BEMF) en la fase que queda libre (en trapezoidal) o mediante observadores matemáticos (en FOC). Simplifica el hardware, pero suele tener problemas de arranque y mal rendimiento a muy bajas revoluciones, ya que sin movimiento no hay BEMF medible.

En el contexto de este trabajo, el hardware utilizado (B-G431B-ESC1) está preparado para ejecutar algoritmos FOC sensorless o sensed, permitiendo evaluar ambas opciones.

2.5 Controladores PID en sistemas dinámicos

2.5.1 Introducción a los controladores PID

El funcionamiento de un controlador PID se basa en actuar sobre el error existente entre una referencia deseada y la salida real del sistema. A partir de este error, el controlador genera una señal de control cuyo objetivo es reducirlo progresivamente a lo largo del tiempo. Este planteamiento encaja especialmente bien en sistemas donde se desea regular una variable concreta, como la velocidad, la posición o la temperatura, incluso cuando existen perturbaciones externas o cambios en las condiciones de funcionamiento.

En el control de motores eléctricos, los PID se utilizan de forma habitual en los lazos de velocidad y de posición. Cuando sus parámetros están bien ajustados, proporcionan una respuesta estable, predecible y suficientemente rápida, lo que explica por qué siguen siendo una herramienta de referencia en este tipo de aplicaciones.

2.5.2 Acción proporcional, integral y derivativa

Un controlador PID se caracteriza por combinar tres tipos de acciones de control diferentes: la proporcional, la integral y la derivativa. Cada una de ellas influye de manera distinta en el comportamiento del sistema cuando se cierra el lazo de control, y su correcta combinación es clave para obtener una respuesta adecuada.

La acción proporcional genera una señal de control directamente relacionada con el valor instantáneo del error. Cuanto mayor es el error, mayor es la acción correctora que se aplica. Este efecto permite aumentar la rapidez de respuesta del sistema, ya que reacciona de forma inmediata ante desviaciones de la referencia. Sin embargo, un control basado únicamente en la acción proporcional suele dar lugar a la aparición de un error en régimen permanente, especialmente en sistemas con cargas o pérdidas constantes.

La acción integral se introduce precisamente para corregir este problema. Su función es acumular el error a lo largo del tiempo, de modo que la señal de control va aumentando progresivamente hasta que el error medio se anula. Gracias a esta acción, es posible eliminar el error en régimen permanente. No obstante, si la acción integral es demasiado intensa, puede provocar respuestas excesivamente lentas u oscilatorias, por lo que su ajuste debe realizarse con cuidado y con criterio.

Por último, la acción derivativa actúa sobre la variación temporal del error, es decir, sobre la rapidez con la que este cambia. De esta forma, el controlador puede anticipar la evolución futura del sistema. Su principal efecto es mejorar la estabilidad y reducir el sobreimpulso, aportando un efecto amortiguador durante el régimen transitorio.

La combinación adecuada de estas tres acciones permite ajustar la respuesta del sistema de forma flexible, logrando un equilibrio entre rapidez de reacción, estabilidad y precisión en el seguimiento de la referencia.

La ley de control de un regulador PID en el dominio del tiempo se expresa como la suma de las tres contribuciones mencionadas:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt} \quad \text{Ecuación (2.10)}$$

Con el objetivo de analizar la estabilidad y diseñar lazos de control, se suele trabajar en el dominio de Laplace, donde la función de transferencia del controlador $C(s)$ se define como:

$$C(s) = K_p + \frac{K_i}{s} + K_d \cdot s$$

Ecuación (2.11)

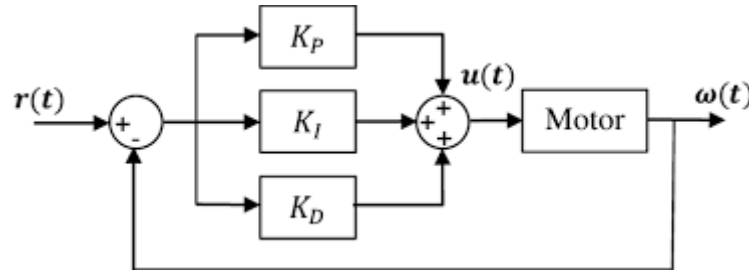


Figura 2.7. Diagrama de bloques control PID. Fuente: International Research Publication House.

2.5.3 Control PID aplicado a sistemas de primer orden

Cuando se aplica un controlador PID a un sistema dinámico de primer orden, como es el caso del modelo adoptado para el conjunto motor–ESC en este trabajo, el análisis del comportamiento en lazo cerrado resulta relativamente directo. Este tipo de sistemas se caracteriza por tener una única constante de tiempo dominante, lo que permite simplificar el diseño mediante técnicas analíticas.

En este escenario, el modelo de la planta (el motor) se define a través de su función de transferencia en el dominio de Laplace, relacionando la tensión de entrada con la velocidad angular obtenida:

$$G_p(s) = \frac{K}{1 + \tau \cdot s}$$

Ecuación (2.12)

Donde K es la ganancia estática y τ es la constante de tiempo del sistema. Para controlar este modelo, se opta por un controlador PI (Proporcional-Integral), cuya estructura resulta muy útil para las posteriores simulaciones:

$$G_{pi}(s) = K_p \cdot \left(1 + \frac{1}{T_i \cdot s} \right) = K_p \cdot \frac{1 + T_i \cdot s}{T_i \cdot s}$$

Ecuación (2.13)

El controlador PID actúa modificando la dinámica natural del sistema con el objetivo de cumplir ciertos criterios de rendimiento. Entre los más habituales se encuentran la reducción del tiempo de establecimiento, la limitación del sobreimpulso y la eliminación del error en régimen permanente. La acción integral pasa a representar un papel clave, ya que permite asegurar que la salida del sistema alcance y mantenga exactamente el valor de referencia deseado, incluso en presencia de perturbaciones o cargas constantes.

El estudio de sistemas de primer orden en lazo cerrado permite, además, establecer relaciones claras y bien definidas entre los parámetros del controlador y la respuesta temporal obtenida. Esta claridad facilita el diseño teórico del regulador y proporciona una base sólida para justificar las decisiones adoptadas durante el ajuste del controlador.

2.5.4 Aplicación del PID al control de velocidad de motores eléctricos

En el control de velocidad de motores eléctricos, el controlador PID suele situarse en el lazo externo de velocidad, actuando sobre una consigna intermedia, normalmente relacionada con el par o la corriente, que es procesada por los lazos internos del sistema. Esta estructura resulta muy eficaz, ya que permite aprovechar la rapidez de respuesta del control de corriente interno mientras el PID se encarga de regular la velocidad de forma estable y precisa. El uso de un controlador PID en este contexto facilita la compensación de perturbaciones externas, como cambios en la carga aplicada al eje, y asegura que la velocidad del motor siga la referencia establecida.

En el sistema desarrollado en este trabajo, el controlador PID se diseña a partir de un modelo dinámico simplificado del conjunto motor-ESC. Este planteamiento permite centrar el diseño del control en la regulación de la velocidad, delegando la gestión de las variables eléctricas más rápidas en el ESC. Como resultado, se obtiene una solución eficaz, coherente con el comportamiento del sistema físico. Además, esta estructura permite que sea fácilmente integrable dentro de la arquitectura distribuida basada en ROS 2 y micro-ROS.

3. Metodología y equipos

3.1 Visión general del sistema desarrollado

El sistema desarrollado en este Trabajo de Fin de Grado tiene como propósito principal implementar un control distribuido de la velocidad de un motor BLDC, combinando un entorno basado en ROS 2 con un sistema embebido real encargado de la actuación física sobre el motor. La idea no es únicamente hacer girar el motor, sino diseñar y validar una arquitectura completa de comunicación y control, muy similar a la que se emplea en sistemas robóticos e industriales reales.

Desde un punto de vista funcional, el sistema permite generar una consigna de velocidad en un nodo ROS 2 que se ejecuta en un ordenador personal y transmitir dicha consigna de manera estructurada y fiable hasta un motor BLDC real. Para lograrlo, se utiliza micro-ROS como elemento de enlace entre el ecosistema ROS 2 y el microcontrolador, que actúa como nodo embebido dentro del sistema distribuido. Además, un controlador electrónico de velocidad (ESC) se encarga del control de bajo nivel del motor, gestionando la electrónica de potencia y la conmutación interna. Esta separación de responsabilidades permite abstraer la complejidad del control eléctrico y centrar el diseño en la comunicación y el control de velocidad.

El flujo general del sistema se puede describir de forma secuencial de la siguiente manera:

ROS 2 → micro-ROS → STM32 → ESC → Motor BLDC

En este esquema, cada bloque cumple una función diferenciada y particular:

- **ROS 2 (PC)** actúa como el nivel de control de alto nivel del sistema. En este entorno se ejecuta un nodo que tiene la función principal de generar y publicar la consigna de velocidad deseada. Su función es totalmente independiente del hardware físico, lo que permite modificar el comportamiento del sistema con mucha facilidad y, además, integrar sin problemas otros nodos o algoritmos más complejos en trabajos posteriores.
- **micro-ROS** hace posible extender el ecosistema ROS 2 hasta un microcontrolador con recursos limitados. Gracias a la presencia del agente micro-ROS, el microcontrolador puede comportarse como un nodo más dentro del sistema distribuido, recibiendo las consignas de velocidad de forma

transparente, sin que el resto de nodos tenga que conocer los detalles del hardware embebido.

- **Microcontrolador STM32** actúa como nexo de unión entre el software y el sistema de potencia. Su función principal es recibir la consigna de velocidad a través de micro-ROS, procesarla y enviarla al controlador del motor mediante un enlace serie UART. En este nivel no se implementa el control eléctrico del motor, sino que se gestiona la comunicación y la lógica de control que se necesita para enlazar cada componente del sistema distribuido general.
- **ESC (Electronic Speed Controller)** es el encargado del control interno del motor BLDC. Este dispositivo gestiona la conmutación de las fases, la modulación PWM y los lazos de control internos, como el control de corriente y, el de velocidad. De esta forma, abstrae toda la complejidad eléctrica asociada al motor. Se puede considerar de esta forma desde el punto de vista del sistema como un actuador inteligente que recibe directamente consignas de velocidad.
- **Motor BLDC** representa el elemento final del sistema, responsable de transformar la energía eléctrica suministrada en movimiento mecánico. Su comportamiento dinámico es el objetivo del control de velocidad implementado a lo largo de toda la arquitectura desarrollada en este trabajo.

Esta separación bien definida de responsabilidades permite abordar el conjunto como un sistema distribuido, en el que cada componente opera en una capa distinta y se comunica con el resto a través de interfaces claras y bien diferenciadas. La verdad es que este planteamiento simplifica notablemente el desarrollo del sistema, ya que cada bloque puede analizarse y ajustarse de forma independiente. Además, refleja una arquitectura realista y muy escalable, permitiendo un posible mayor desarrollo o ampliación del trabajo en un futuro.

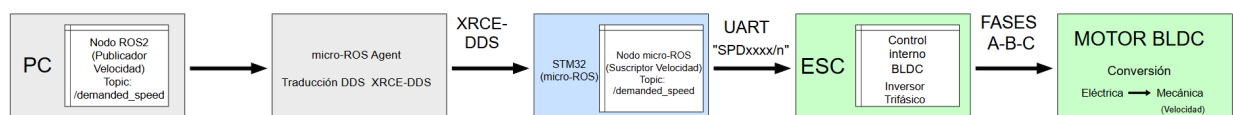


Figura 3.1. Diagrama de bloques del sistema desarrollado. Fuente: Propia.

3.2 Arquitectura del sistema distribuido

El sistema desarrollado en este trabajo se ha creado siguiendo una arquitectura distribuida, en la que las distintas funciones se reparten entre varios bloques bien diferenciados, tanto desde el punto de vista del software como del hardware. En lugar de concentrar todo el procesamiento y la lógica de control en un único dispositivo, se opta por dividir el sistema en varios niveles que cooperan entre sí. La verdad es que esta separación resulta muy ventajosa: cada bloque puede centrarse en una tarea concreta, lo que simplifica el diseño global y hace que el sistema sea más fácil de desarrollar, depurar y mantener con el paso del tiempo. Además, este enfoque permite modificar o ampliar partes concretas del sistema sin necesidad de rehacer toda la arquitectura.

De forma general, la arquitectura del sistema se organiza en tres niveles principales. En primer lugar, un nivel de control de alto nivel basado en ROS 2, encargado de generar y gestionar las consignas de velocidad. En segundo lugar, un nivel de control embebido implementado sobre un microcontrolador STM32 mediante micro-ROS, que actúa como puente entre el entorno ROS y el hardware. Por último, un nivel de potencia y control del motor BLDC gestionado por el ESC, responsable de la actuación física sobre el motor y de los lazos de control de bajo nivel.

3.2.1 Justificación de la arquitectura distribuida

La elección de una arquitectura distribuida se apoya, principalmente, en la separación natural de funciones entre los distintos niveles de control que componen el sistema. Cada uno de estos niveles trabaja con información diferente y opera en escalas temporales distintas, por tanto, concentrarlos en una única unidad de control generaría el efecto contrario al deseado, aumentar tremendamente la complejidad del sistema y su funcionamiento.

Control de alto nivel (ROS 2)

El control de alto nivel se ejecuta en un ordenador personal dentro del entorno ROS 2. En este nivel se generan las consignas de velocidad del motor y se gestiona la lógica global del sistema. ROS 2 tiene un marco de trabajo muy flexible para la comunicación entre nodos, lo que permite, modificar el comportamiento del sistema, añadir nuevas funcionalidades o integrar otros dispositivos sin alterar la estructura general de la arquitectura.

Además, este nivel se mantiene alejado del hardware de potencia y de los detalles eléctricos del motor. Trabaja con variables abstractas, como la velocidad deseada, lo

cual encaja muy bien con el enfoque modular que se ha utilizado en este trabajo y favorece la escalabilidad del sistema.

Control embebido (STM32 con micro-ROS)

El nivel intermedio del sistema se implementa sobre un microcontrolador STM32, que actúa como enlace entre el entorno ROS 2 y el sistema de control del motor. Gracias a micro-ROS, el microcontrolador puede integrarse en el ecosistema ROS 2, suscribiéndose a los tópicos de velocidad generados en el nivel superior y recibiendo las consignas de manera transparente.

En este nivel se concentran las tareas típicas de un sistema embebido: la recepción de comandos, su interpretación y la generación de señales de control que se envían al ESC a través de comunicación serie UART. El uso de micro-ROS resulta especialmente útil, ya que permite mantener el modelo de comunicación de ROS sin exigir al microcontrolador los recursos necesarios para ejecutar un nodo ROS 2 completo.

Control de potencia (ESC)

El nivel más bajo de la arquitectura corresponde al control de potencia del motor BLDC, que se delega por completo en el ESC B-G431B-ESC1. Este dispositivo se encarga de la conmutación de las fases del motor, la modulación PWM, la estimación de la posición del rotor y la ejecución de los lazos internos de corriente y par [17].

Desde el punto de vista del sistema distribuido, el ESC encapsula toda la complejidad eléctrica asociada al motor y permite tratarlo como un actuador de velocidad. Esto simplifica de forma notable el diseño del control de nivel superior, ya que no es necesario implementar directamente algoritmos de control de bajo nivel en el microcontrolador principal, manteniendo así una arquitectura más limpia, robusta y fácil de mantener.

Ventajas de la arquitectura distribuida

La adopción de una arquitectura distribuida aporta una serie de ventajas que han sido determinantes en el diseño del sistema desarrollado en este trabajo. En primer lugar, destaca la modularidad del conjunto: cada bloque cumple una función bien delimitada, lo que permite modificarlo, ampliarlo o incluso sustituirlo sin necesidad de replantear el sistema completo.

Otra ventaja muy relevante es la escalabilidad. Al apoyarse en ROS 2 y en un esquema de comunicación bien estructurado, la arquitectura permite incorporar nuevos nodos, sensores o actuadores de forma relativamente sencilla. De esta manera, el sistema puede crecer y adaptarse a nuevas necesidades manteniendo siempre una buena organización.

Es una ventaja enorme el distribuir las responsabilidades entre distintos niveles, ya que los errores o modificaciones introducidos en uno de ellos no afectan directamente al resto. Esto simplifica las tareas de mantenimiento o búsqueda de posibles errores, y permite aislar problemas de manera más clara y eficiente.

Por último, también es importante destacar el realismo industrial de la solución escogida. La estructura propuesta es muy similar a la que se emplea en entornos industriales reales, donde el control de alto nivel, los sistemas embebidos y la electrónica de potencia suelen estar claramente diferenciados y comunicarse mediante interfaces bien definidas.

En conjunto, esta arquitectura distribuida constituye una base sólida y coherente para el desarrollo del sistema presentado, y justifica plenamente el uso combinado de ROS 2, micro-ROS, microcontroladores STM32 y un ESC dedicado para el control de velocidad del motor BLDC.

3.2.2 Nodos y responsabilidades del sistema

Una vez establecida la arquitectura distribuida del sistema, es necesario bajar un nivel más y concretar qué elementos intervienen realmente y qué papel desempeña cada uno. En este proyecto, el funcionamiento global se articula en torno a tres entidades principales: un nodo ROS 2 que se ejecuta en el PC, un nodo micro-ROS desplegado en el microcontrolador STM32 y el ESC, que actúa como un actuador inteligente encargado del control del motor BLDC.

Esta separación permite asignar a cada bloque tareas acordes a sus capacidades reales, tratando de evitar cargar de complejidad innecesaria a los niveles más bajos del sistema y manteniendo una estructura clara y comprensible.

Nodo ROS 2 en el PC (publicador de consigna)

El nodo ROS 2 que se ejecuta en el ordenador personal representa el nivel de control más alto del sistema. Su función principal es generar la consigna de velocidad del motor y publicarla dentro del ecosistema ROS 2 a través de un tópico específico, denominado `/darjan/demanded_speed`.

Este nodo actúa, como el origen de la referencia de control. Una de sus principales ventajas es la flexibilidad con la que puede ser implementada: la consigna de velocidad puede proceder de una entrada manual, de una lógica de control más elaborada, de una interfaz gráfica o incluso de otros nodos ROS que formen parte de un sistema de mayor envergadura. Gracias al enfoque modular de ROS 2, este nodo

puede modificarse, ampliarse o sustituirse sin que ello implique cambios en el resto de la arquitectura.

Desde un punto de vista funcional, el nodo ROS 2 permanece completamente ajeno a los detalles de bajo nivel del sistema. No tiene conocimiento alguno sobre cómo se controla el motor, cómo se realiza la comunicación con el hardware ni qué electrónica interviene en el proceso. Su responsabilidad se limita a publicar una referencia de velocidad en términos abstractos, delegando por completo la ejecución física de esta consigna en los niveles inferiores del sistema.

Nodo micro-ROS en el STM32

El nodo micro-ROS desplegado en el microcontrolador STM32 actúa como el punto de enlace entre el ecosistema ROS 2 y el hardware embebido del sistema. Su función principal es suscribirse al tópico `/darjan/demanded_speed`, publicado desde el nodo ROS 2 que se ejecuta en el PC, y recibir a través del agente micro-ROS las consignas de velocidad generadas en el nivel superior.

Una vez que la consigna llega al microcontrolador, este se encarga de interpretarla y adaptarla al formato que necesita el controlador de motor. En el caso de este proyecto, esta adaptación se realiza mediante el envío de comandos a través de una interfaz UART, utilizando un formato de mensaje sencillo y fácilmente reconocible por el ESC. De este modo, se establece un enlace claro y directo entre la referencia generada en ROS 2 y el sistema encargado de la actuación física.

Es importante destacar que el nodo micro-ROS no asume tareas complejas de control del motor ni interviene en la gestión directa de las fases del BLDC. Su papel se limita a la gestión de la comunicación y a la traducción de consignas. Este planteamiento permite mantener el código embebido relativamente simple, robusto y adecuado para ejecutarse en un microcontrolador de estas características, evitando sobrecargarlo con algoritmos que no le corresponden.

ESC como actuador inteligente

El ESC B-G431B-ESC1 desempeña el papel de actuador inteligente dentro del sistema distribuido. A diferencia de un actuador pasivo, este dispositivo integra tanto la electrónica de potencia necesaria como algoritmos de control avanzados que le permiten gestionar internamente la conmutación de las fases del motor, la modulación PWM y los lazos de control de corriente y par.

Desde el punto de vista del resto del sistema, el ESC se limita a recibir una consigna de velocidad o de control a través de la interfaz UART y a ejecutarla sobre el motor BLDC real. Esto implica que ni el microcontrolador ni el nodo ROS 2 necesitan conocer

los detalles eléctricos del motor ni implementar estrategias de conmutación o control de bajo nivel, ya que toda esa complejidad queda encapsulada dentro del propio ESC.

Gracias a esta abstracción, el motor BLDC puede tratarse, a efectos prácticos, como un actuador de velocidad. Este enfoque simplifica notablemente el diseño global del sistema y permite centrar el trabajo en aspectos como la comunicación, la integración de micro-ROS y el control de alto nivel, manteniendo una arquitectura clara, coherente y fácilmente extensible.

3.2.3 Flujo de datos extremo a extremo

Una vez definidas la arquitectura distribuida del sistema y las responsabilidades de cada nodo, resulta necesario describir de una forma precisa cómo fluye la información desde que se genera una consigna de velocidad en el entorno ROS 2 hasta que esta se traduce en un movimiento físico del motor BLDC. Este análisis extremo a extremo permite entender el funcionamiento global del sistema y verificar que cada capa cumple correctamente su función.

El flujo de datos se organiza de forma secuencial y jerárquica, atravesando distintos niveles de abstracción, desde software de alto nivel hasta electrónica de potencia.

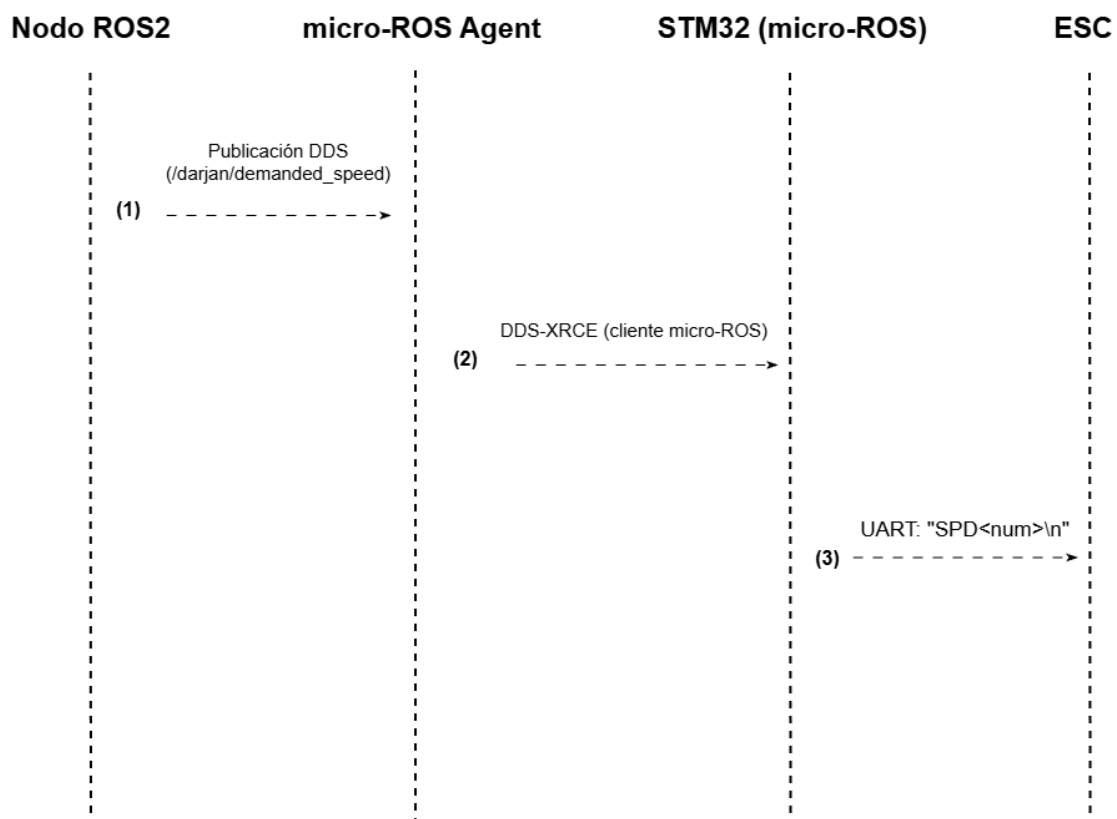


Figura 3.2. Diagrama de flujo de datos en la arquitectura distribuida del sistema. Fuente: Propia.

Publicación de la consigna en el tópico `/darjan/demanded_speed`

El flujo de información comienza en el nodo ROS 2 que se ejecuta en el ordenador personal, el cual asume el papel de generador de la referencia de velocidad. Este nodo publica de forma periódica mensajes en el tópico `/darjan/demanded_speed`, donde se encapsula el valor de velocidad deseado para el motor.

La publicación se realiza siguiendo el modelo publish/subscribe característico de ROS 2. Gracias a este mecanismo, cualquier nodo que lo necesite puede suscribirse al tópico sin que exista una dependencia directa con el nodo publicador. En el contexto de este trabajo, el valor transmitido representa una consigna de velocidad a nivel abstracto, completamente desacoplada del hardware y de los detalles físicos del sistema.

Este planteamiento aporta una gran flexibilidad al diseño. La fuente de la consigna puede modificarse fácilmente en el futuro sin que sea necesario alterar el resto de la arquitectura. De esta forma, el sistema queda preparado para evolucionar sin perder coherencia.

Transporte DDS y comunicación a través de micro-ROS

Una vez publicada la consigna, la infraestructura de comunicación de ROS 2, basada en el estándar DDS (Data Distribution Service), se encarga de propagar el mensaje a través del sistema. No obstante, dado que el nodo embebido no se conecta directamente a una red DDS completa, es necesario introducir un mecanismo intermedio: micro-ROS [3].

En este punto entra en acción el agente micro-ROS, que actúa como puente entre el ecosistema ROS 2 y el microcontrolador. El agente recibe el mensaje publicado en el tópico `/darjan/demanded_speed` y lo adapta al protocolo DDS-XRCE, una versión ligera del estándar DDS específicamente diseñada para entornos con recursos limitados.

Gracias a esta traducción, el nodo micro-ROS que se ejecuta en el STM32 puede recibir la consigna de velocidad de manera transparente, como si formara parte de un sistema ROS 2 convencional. De esta forma, se puede mantener la coherencia del sistema distribuido sin exigir al microcontrolador la complejidad asociada a una pila de comunicación completa.

Cuando el nodo micro-ROS que se ejecuta en el STM32 recibe el mensaje de velocidad, este se gestiona directamente a nivel de aplicación. En esta etapa, la consigna debe adaptarse al formato que el ESC es capaz de interpretar correctamente, actuando así como un puente entre ROS y el controlador de potencia.

En el sistema desarrollado, esta adaptación se realiza generando un comando en formato de texto sencillo y estructurado, siguiendo el patrón:

`SPD<num>\n`

donde `<num>` corresponde al valor numérico de la velocidad deseada. Este mensaje se envía al ESC a través de la interfaz UART previamente configurada en el microcontrolador.

La elección de un formato textual claro facilita claramente la verificación del sistema, especialmente durante las fases de desarrollo y pruebas. Además, permite mantener una separación clara entre la lógica de control de alto nivel y el protocolo de comunicación empleado.

Interpretación de la consigna por el ESC y actuación sobre el motor

Una vez recibido el comando por UART, el ESC lo interpreta como una referencia de control. A partir de esta información, el controlador de potencia ejecuta todas las tareas asociadas al control de bajo nivel del motor BLDC.

Este proceso abarca la gestión del inversor trifásico, la generación de las señales PWM, la conmutación electrónica de las fases del motor y la ejecución de los lazos internos de control. Como resultado de estas acciones, el motor BLDC ajusta su velocidad para seguir la consigna enviada desde el entorno de ROS 2.

Desde la perspectiva global del sistema distribuido, el ESC se comporta como un bloque que encapsula por completo la complejidad del control eléctrico del motor. Esto permite que los niveles superiores trabajen exclusivamente con magnitudes de alto nivel, como la velocidad deseada, simplificando de esta manera el diseño y reforzando la modularidad de la arquitectura del sistema.

3.3 Hardware utilizado

3.3.1 Microcontrolador principal (STM32 NUCLEO-F446ZE)

El microcontrolador principal empleado en el sistema desarrollado es la placa STM32 NUCLEO-F446ZE, basada en el microcontrolador STM32F446ZET6 de STMicroelectronics. Esta placa representa el núcleo del sistema embebido y es la encargada de ejecutar el nodo micro-ROS, gestionar la comunicación con el entorno ROS 2 y transmitir las consignas de control al controlador electrónico de potencia (ESC) [16].

La elección de la plataforma a utilizar era de vital importancia para poder cumplir tanto con los requisitos funcionales del proyecto como con criterios de escalabilidad y realismo industrial. Se buscaba una solución que permitiera integrar de forma fiable un sistema distribuido basado en ROS 2, sin introducir complejidades innecesarias a nivel de hardware.

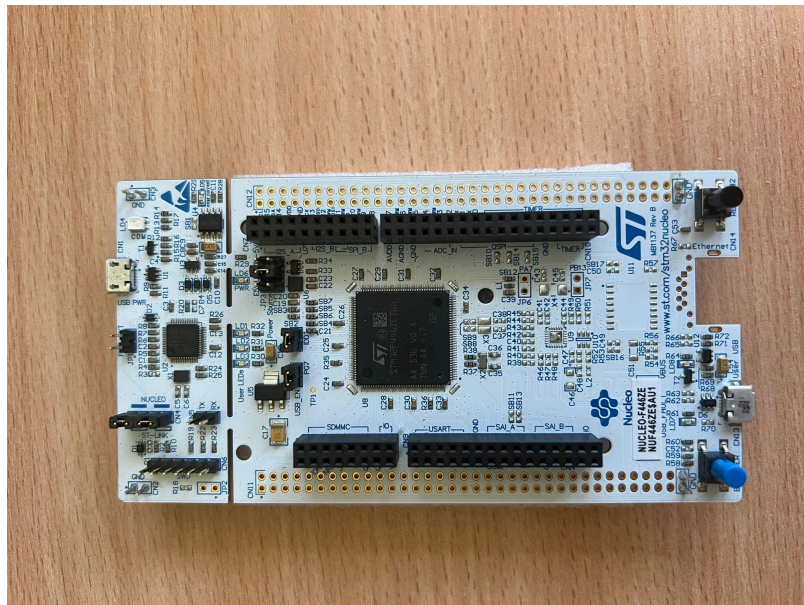


Figura 3.3. Placa de desarrollo NUCLEO-F446ZE utilizada. Fuente: Propia.

Justificación de la elección

Uno de los motivos clave para seleccionar el STM32F446 es su compatibilidad completa con micro-ROS. Este microcontrolador dispone de recursos suficientes para ejecutar un sistema operativo en tiempo real como FreeRTOS, algo fundamental para

coordinar las tareas de comunicación, recepción de consignas y envío de comandos al ESC. Además, la placa NUCLEO-F446ZE es una plataforma ampliamente utilizada tanto en entornos académicos como industriales. Gracias a eso, hay una gran disponibilidad de documentación, ejemplos y soporte, facilitando así el desarrollo.

Otro factor relevante para la elección de este microcontrolador es la presencia de múltiples interfaces de comunicación, especialmente UART/USART. Estas interfaces son esenciales para realizar la comunicación serie con el ESC y permiten, además, separar distintos canales de comunicación. Esta flexibilidad resulta muy útil tanto durante el desarrollo como en posibles ampliaciones futuras del sistema [16].

Por último, el STM32F446 ofrece un equilibrio adecuado entre potencia de cálculo y consumo energético. Es capaz de ejecutar tareas relacionadas con ROS 2, el procesamiento de mensajes y la lógica de control sin necesidad de recurrir a microcontroladores de un coste mas elevado. Este equilibrio refuerza el enfoque realista creado respecto al sistema desarrollado.

Recursos relevantes utilizados en el proyecto

El microcontrolador STM32F446 integra un núcleo ARM Cortex-M4 con unidad de coma flotante (FPU), lo que permite realizar cálculos matemáticos y operaciones temporales de forma eficiente. Aunque en este proyecto el control eléctrico de bajo nivel del motor se delega en el ESC, disponer de esta capacidad aporta margen y solidez al sistema embebido, especialmente de cara a la gestión de tareas y comunicaciones [16].

Entre los recursos más relevantes empleados en el desarrollo del proyecto destacan los siguientes:

- **Interfaz UART:** se utiliza como medio principal para la transmisión de consignas de velocidad desde el microcontrolador hacia el ESC. La comunicación se basa en tramas de texto con un formato bien definido, lo que no solo facilita la interoperabilidad entre distintos dispositivos, sino que también es muy práctico durante las fases de revisión y depuración de errores, ya que permite observar y verificar de forma rápida y visual los mensajes transmitidos.
- **FreeRTOS:** utilizado como sistema operativo en tiempo real para organizar la ejecución de tareas. Permite separar claramente la lógica de comunicación micro-ROS, la recepción de consignas y la transmisión por UART, garantizando

de esta forma un comportamiento estable.

- **Timers y periféricos internos:** aunque no se emplean directamente para el control del motor, los temporizadores del microcontrolador son esenciales para la gestión temporal del sistema, el planificador del RTOS y la sincronización de tareas.
- **Conectividad con el entorno de desarrollo:** La placa NUCLEO integra un programador y depurador ST-LINK, lo que simplifica enormemente la carga de firmware, la ejecución paso a paso y la depuración del código durante el desarrollo del proyecto.

En conjunto, el STM32 NUCLEO-F446ZE se adapta de manera muy adecuada a los objetivos planteados, actuando como un puente eficiente entre el entorno ROS 2 de alto nivel y el control embebido necesario para interactuar con el hardware de potencia del sistema.

3.3.2 Controlador electrónico de velocidad (ESC B-G431B-ESC1)

El control de potencia del motor BLDC se lleva a cabo mediante el Electronic Speed Controller (ESC) B-G431B-ESC1, una placa de desarrollo de STMicroelectronics creada específicamente para aplicaciones de control de motores trifásicos sin escobillas. Este dispositivo es el encargado de gestionar toda la electrónica asociada al accionamiento del motor: desde la conmutación de las fases hasta la generación de las señales PWM y la ejecución de los lazos internos de control de par y velocidad. En la práctica, el ESC actúa como el vínculo directo entre el sistema embebido y el motor eléctrico [17].

La placa B-G431B-ESC1 está basada en el microcontrolador STM32G431, perteneciente a la familia STM32G4, orientada específicamente a aplicaciones de control de potencia y motores eléctricos. Este microcontrolador integra periféricos pensados para este tipo de tareas, como temporizadores avanzados para la generación de PWM, convertidores analógico-digitales de alta velocidad y comparadores analógicos. Gracias a estos recursos, el ESC puede implementar estrategias de control con tiempos de respuesta muy reducidos [17].

Además del microcontrolador, el ESC incorpora un inversor trifásico de potencia, formado por un conjunto de transistores MOSFET. Este inversor es el encargado de aplicar las tensiones adecuadas a las tres fases del motor, permitiendo generar un campo magnético giratorio de forma precisa y controlada. La estrecha integración entre el microcontrolador y la etapa de potencia hace posible un control eficiente y

estable del motor BLDC, abstraendo toda esta complejidad del resto del sistema [17].

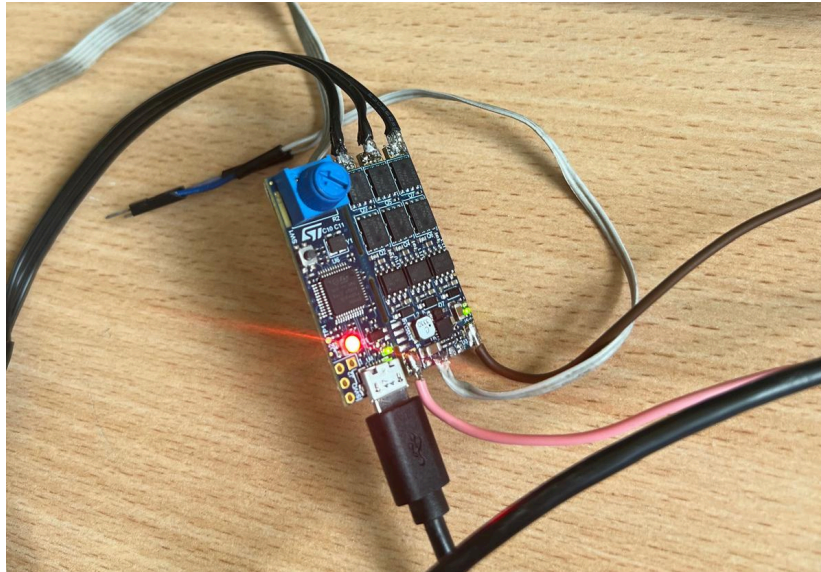


Figura 3.4. Controlador electrónico de velocidad (ESC) B-G431B-ESC1. Fuente: Propia.

Control interno del motor BLDC

Uno de los aspectos más destacables del ESC B-G431B-ESC1 es que integra internamente todo el control del motor BLDC. Esto incluye tanto estrategias de conmutación en seis pasos como técnicas más avanzadas de control vectorial (FOC) en configuración sensorless. En la práctica, esto significa que el propio ESC es capaz de estimar la posición del rotor, regular las corrientes de fase y ajustar la modulación PWM sin necesidad de sensores externos adicionales [17]. Desde el punto de vista del sistema desarrollado, esta característica es muy relevante, ya que permite abstraer por completo la complejidad eléctrica asociada al motor. El microcontrolador STM32F446 no tiene que preocuparse por la gestión directa de las fases, ni por el cálculo de señales PWM o la sincronización con la posición del rotor. Su función se limita a enviar consignas de velocidad al ESC, delegando todo el control de bajo nivel en un dispositivo específicamente diseñado para ello.

Interfaz de comunicación con el microcontrolador

La comunicación entre el STM32 principal y el ESC se realiza mediante una interfaz UART, utilizando un protocolo de texto sencillo. Las consignas de velocidad se envían en forma de cadenas con el formato:

SPD<num>\n

donde `<num>` representa la velocidad deseada del motor expresada en revoluciones por minuto (rpm). Este enfoque, es muy práctico: facilita la implementación del sistema, simplifica la depuración de errores durante el desarrollo y es suficiente para transmitir consignas de control de forma fiable.

El ESC interpreta estas tramas y ajusta internamente su lazo de control para modificar la velocidad del motor en consecuencia. De este modo, el ESC actúa como un actuador inteligente, capaz de recibir comandos de alto nivel y traducirlos en acciones eléctricas complejas sobre el motor BLDC, sin que el resto del sistema tenga que lidiar con esos detalles.

Papel del ESC dentro del sistema distribuido

Dentro de la arquitectura distribuida planteada en este trabajo, el ESC ocupa un papel muy concreto y bien delimitado como módulo responsable del control de potencia y de la ejecución del control de bajo nivel del motor. Mientras que ROS 2 y micro-ROS se encargan de la generación de consignas y de la comunicación entre los distintos bloques del sistema, es el ESC quien garantiza que el motor BLDC funcione de manera estable, eficiente y segura en todo momento.

Además, este enfoque modular aporta ventajas claras desde el punto de vista del diseño y la evolución del sistema. Por un lado, facilita la sustitución del motor o la modificación de la estrategia de control interna del ESC sin necesidad de realizar cambios profundos en el resto de la arquitectura. Por otro, incrementa la robustez global del sistema y simplifica el desarrollo del software embebido, al reducir su complejidad.

3.3.3 Motor BLDC utilizado

El motor utilizado para el sistema ha sido un Maxon EC motor, un motor sin escobillas (brushless) de corriente continua, fabricado por la empresa suiza Maxon, especializada en soluciones de precisión para el sector de la automatización, la robótica o la industria médica. Este tipo de motores, también conocidos como BLDC (Brushless DC Motor), funcionan mediante la conmutación electrónica de las fases y no requieren escobillas físicas, permitiendo por tanto una mayor eficiencia, menor mantenimiento y una vida útil superior a los motores DC convencionales [23].

Este motor está identificado con el número de referencia 251601 / 198692, y está diseñado para funcionar junto a un sistema de control trifásico, como el que proporciona la placa B-G431B-ESC1 empleada en este proyecto. La elección de un

motor de estas características se justifica por su capacidad de respuesta, su comportamiento predecible y su buena integración con drivers de control vectorial [23].

Características generales del motor BLDC

Desde el punto de vista constructivo, el motor utilizado en este trabajo tiene una estructura trifásica, con un estator bobinado en tres fases y un rotor equipado con imanes permanentes. La conmutación de las corrientes en los devanados del estator no se realiza de forma mecánica, sino electrónicamente a través del ESC, lo que permite un control más preciso y eficiente del motor [23].

El par electromagnético generado es aproximadamente proporcional a la corriente que circula por las fases del estator, mientras que la velocidad de giro viene determinada por el equilibrio entre el par, las pérdidas mecánicas internas y la carga aplicada al eje. Esta relación es especialmente adecuada para su control usando estrategias basadas en la regulación de corriente y velocidad, como las empleadas en este proyecto [23].

Además, el motor presenta una inercia mecánica considerablemente baja (*Figura 3.7*), lo que implica respuestas rápidas ante variaciones en la consigna de velocidad. Esto es especialmente importante en el contexto del sistema desarrollado, debido a que facilita la obtención de un comportamiento dinámico y estable incluso cuando el control de velocidad se realiza desde un lazo externo de alto nivel [23].

Integración con el ESC

En el sistema desarrollado, el motor no se controla de forma directa desde el microcontrolador principal, sino que su gestión queda completamente delegada en el ESC. Es este dispositivo el que se encarga de generar las señales PWM necesarias, estimar la posición del rotor y regular las corrientes de fase, implementando internamente los algoritmos de control típicos de un motor BLDC.

Desde un punto de vista funcional, esta decisión permite considerar el conjunto formado por el ESC y el motor como un único subsistema. A efectos del control de alto nivel, dicho conjunto puede aproximarse al comportamiento de un actuador de velocidad, lo que simplifica enormemente la interacción con el resto del sistema. El microcontrolador STM32F446 se limita a enviar consignas de velocidad al ESC, sin necesidad de gestionar directamente ni los fenómenos eléctricos ni los aspectos mecánicos internos del motor.

Papel del motor en el sistema global

Dentro de la arquitectura distribuida del sistema, el motor BLDC representa el último bloque de la cadena de control. Las consignas de velocidad generadas en el entorno ROS 2 se propagan a través de micro-ROS y del sistema embebido hasta llegar al ESC, que finalmente actúa sobre el motor para ajustar su velocidad y cumplir la referencia establecida.

El comportamiento dinámico del motor, en combinación con los lazos de control internos implementados en el ESC, condiciona directamente la respuesta global del sistema. Por tanto, el motor se considera un elemento clave en el análisis del control de velocidad, y su dinámica se tiene en cuenta posteriormente tanto en el modelado del sistema como en el diseño del controlador PID de alto nivel.

En conjunto, el motor BLDC utilizado actúa como un actuador eficiente, robusto y bien integrado, cuya complejidad interna queda encapsulada por el ESC. Esto permite centrar el desarrollo del trabajo en los aspectos de comunicación distribuida, control de alto nivel e integración entre software y hardware, sin renunciar a un comportamiento realista.

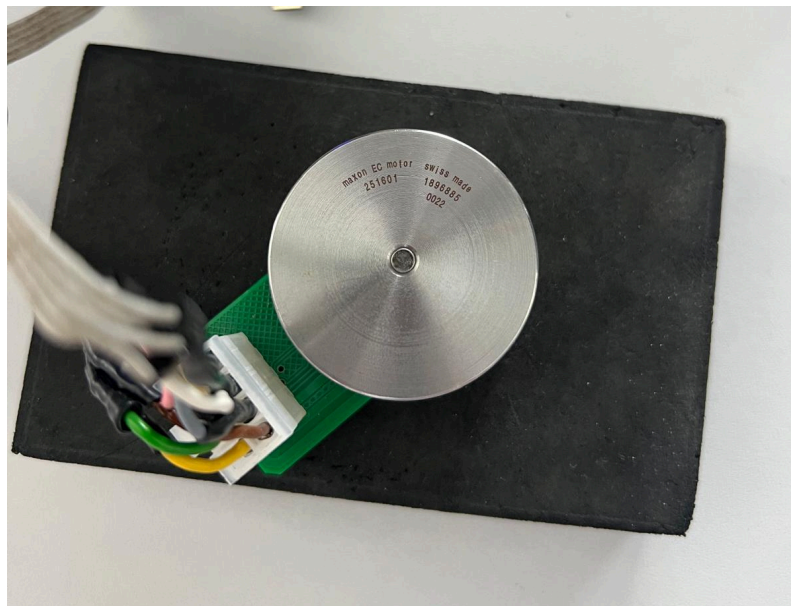
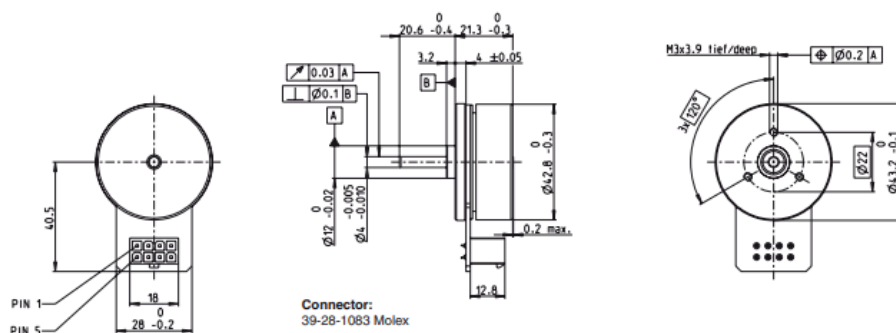


Figura 3.5. Motor BLDC utilizado en las pruebas experimentales. Fuente: Propia.

EC 45 flat Ø42.8 mm, brushless, 50 Watt

maxon flat motor



M 1:2

Figura 3.6. Datos técnicos Maxon EC Motor 251601 Fuente: Maxon Motor [23].

with Hall sensors		251601
Motor Data		
Values at nominal voltage		
1 Nominal voltage	V	24
2 No load speed	rpm	6710
3 No load current	mA	185
4 Nominal speed	rpm	5250
5 Nominal torque (max. continuous torque)	mNm	82.7
6 Nominal current (max. continuous current)	A	2.32
7 Stall torque	mNm	741
8 Starting current	A	23.3
9 Max. efficiency	%	83
Characteristics		
10 Terminal resistance phase to phase	Ω	1.03
11 Terminal inductance phase to phase	mH	0.572
12 Torque constant	mNm/A	33.5
13 Speed constant	rpm/V	285
14 Speed/torque gradient	rpm/mNm	8.75
15 Mechanical time constant	ms	12.4
16 Rotor inertia	gcm ²	135

Figura 3.7. Datos técnicos Maxon EC Motor 251601 Fuente: Maxon Motor [23]

3.3.4 Fuente de alimentación programable

Para alimentar el subsistema de potencia se ha empleado una fuente de alimentación de laboratorio programable, encargada de suministrar la tensión continua necesaria tanto para el funcionamiento del ESC como para el motor BLDC. Esta elección permite trabajar en un entorno controlado y seguro, algo especialmente importante durante las fases iniciales de pruebas y puesta en marcha.

El uso de una fuente de laboratorio, en lugar de una batería u otra fuente fija, aporta una ventaja clara en términos de control y repetibilidad. Permite definir con precisión las condiciones de operación y repetir los ensayos bajo los mismos parámetros, algo muy importante en un trabajo experimental. Además, ofrece la posibilidad de variar la

tensión de alimentación de forma controlada. Esto puede resultar útil para analizar cómo influye este parámetro en la respuesta dinámica del conjunto motor–ESC.

En el contexto de este proyecto, la fuente de alimentación actúa como el origen de energía del subsistema de potencia, mientras que el resto del sistema se alimenta de manera independiente a través de la interfaz USB del ordenador. Esta separación contribuye a un funcionamiento más estable y facilita el aislamiento de posibles problemas durante las pruebas y por tanto su fácil identificación.



Figura 3.8. Fuente de alimentación programable configurada para el sistema. Fuente: Propia.

3.3.5 Instrumentación de medida y depuración (osciloscopio)

Para la observación directa y la verificación del comportamiento eléctrico del sistema se ha utilizado un osciloscopio de laboratorio, especialmente durante las etapas de depuración, búsqueda de posibles errores y validación del funcionamiento global. Esta herramienta ha permitido ir más allá de lo que puede observarse únicamente mediante software, ofreciendo una visión clara y en tiempo real de lo que realmente ocurre a nivel de señales.

El osciloscopio se empleó para analizar distintos elementos clave del sistema, como las líneas de comunicación UART entre el microcontrolador STM32 y el ESC, así como señales de control y temporización relevantes para el correcto funcionamiento del conjunto. También fue muy útil para observar el comportamiento temporal de las señales durante cambios en la consigna de velocidad, permitiendo comprobar cómo se

propagaban estos cambios a través del sistema.

El uso del osciloscopio es, en este sentido, una herramienta fundamental en el desarrollo de sistemas embebidos, ya que complementa la información obtenida por programación y facilita la detección de fallos. Su uso en este trabajo ha sido clave para asegurar que el sistema se comportaba de forma correcta.



Figura 3.9. Osciloscopio digital empleado para la verificación de señales. Fuente: Propia.

3.4 Software utilizado

El desarrollo del sistema presentado en este trabajo ha requerido el uso de distintas herramientas software, cada una orientada a una parte concreta del problema. Por un lado, se ha trabajado con entornos de alto nivel para la comunicación y el control distribuido; por otro, con herramientas para el desarrollo embebido, el control de motores y la simulación del comportamiento dinámico del sistema. La combinación de todas ellas ha sido imprescindible para la total validación del sistema.

De manera general, el software empleado puede agruparse en cuatro bloques principales: el entorno ROS 2 ejecutado en el PC, la extensión micro-ROS para sistemas embebidos, las herramientas de desarrollo para microcontroladores STM32 y las aplicaciones utilizadas para el control y la simulación del motor BLDC.

3.4.1 ROS 2 en el PC

El nivel de control de alto nivel del sistema se ha implementado utilizando ROS 2, que

actúa como plataforma central para la generación y distribución de consignas de control. Se ha desarrollado un nodo ROS 2 encargado de publicar la velocidad deseada del motor a través del tópico `/darjan/demanded_speed`.

Este nodo se ejecuta en un ordenador personal y hace uso del modelo publish/subscribe característico de ROS 2, lo que permite por tanto, separar claramente la generación de la consigna de velocidad del hardware que finalmente la ejecuta. La verdad es que esta forma de trabajar aporta mucha flexibilidad: la lógica de control puede modificarse, ampliarse o incluso sustituirse sin necesidad de realizar cambios en el sistema embebido ni en el controlador del motor.

3.4.2 micro-ROS en el sistema embebido

Para integrar el microcontrolador STM32 dentro del ecosistema ROS 2 se ha recurrido a micro-ROS, una extensión pensada específicamente para llevar las capacidades de ROS 2 a sistemas embebidos con recursos limitados. Gracias a micro-ROS, es posible ejecutar nodos compatibles con ROS 2 directamente sobre un microcontrolador, manteniendo la misma estructura de nodos, tópicos y mensajes que en un sistema ROS convencional.

En este proyecto, el microcontrolador ejecuta un nodo micro-ROS cuya función principal es suscribirse al tópico `/darjan/demanded_speed`. A través de este nodo se reciben las consignas de velocidad generadas en el PC. La comunicación con el resto del sistema ROS 2 no se realiza de forma directa, sino mediante un agente micro-ROS, que actúa como intermediario y se encarga de traducir los mensajes DDS al protocolo ligero DDS-XRCE, mucho más adecuado para microcontroladores.

La verdad es que este planteamiento encaja muy bien con los objetivos del trabajo. Permite integrar el sistema embebido como una parte más del sistema distribuido y simplifica el desarrollo y la depuración del sistema. Además, consolida la coherencia global de la arquitectura, ya que todos los elementos comparten el mismo modelo de comunicación.

3.4.3 Entorno de desarrollo embebido (STM32CubeIDE y FreeRTOS)

El desarrollo del software embebido se ha realizado utilizando STM32CubeIDE, el entorno de desarrollo integrado proporcionado por STMicroelectronics. Esta herramienta ha servido como base para la configuración inicial del microcontrolador, la

activación de los periféricos necesarios y la generación del código base sobre el que se ha construido la aplicación [18].

Sobre este entorno se ha integrado FreeRTOS como sistema operativo en tiempo real, lo que permite estructurar el software en tareas independientes y garantizar un comportamiento predecible del sistema. En este proyecto, FreeRTOS se emplea para separar claramente distintas responsabilidades, como la gestión de la comunicación micro-ROS, el tratamiento de las consignas de velocidad recibidas y el envío de comandos al ESC a través de la interfaz UART [24].

Esta organización resulta muy útil en un sistema distribuido como este, ya que facilita la sincronización entre tareas, mejora la legibilidad del código y permite escalar el sistema en el futuro manteniéndose estable. En conjunto, la combinación de STM32CubeIDE y FreeRTOS aporta un entorno sólido y flexible para implementar los sistemas embebidos necesarios en este trabajo [18,24].

3.4.4 Motor Control Workbench y configuración del ESC

La configuración y del ESC B-G431B-ESC1 se ha realizado utilizando Motor Control Workbench, una herramienta desarrollada por STMicroelectronics orientada al diseño, ajuste y validación de sistemas de control de motores eléctricos. Esta aplicación permite trabajar de forma relativamente directa con motores BLDC reales, abstraendo gran parte de la complejidad asociada al control de bajo nivel [19].

Motor Control Workbench ofrece la posibilidad de seleccionar distintas estrategias de control, como la conmutación en seis pasos o el control vectorial orientado al campo (FOC), además de introducir los parámetros eléctricos y mecánicos del motor BLDC utilizado. A partir de esta información, la herramienta genera automáticamente el firmware que se ejecuta en el microcontrolador integrado en el ESC, configurando los lazos internos de control de corriente, par y velocidad [18,19].

La verdad es que el uso de Motor Control Workbench encaja especialmente bien con el enfoque de este trabajo. Permite delegar completamente el control eléctrico del motor en un sistema probado, evitando la necesidad de implementar manualmente algoritmos complejos de control de motores. De esta manera, el ESC se comporta como un actuador inteligente, mientras que el resto del sistema puede centrarse en la comunicación y el control de alto nivel.

3.4.5 Simulink y Simscape para simulación y diseño del control

Para el análisis dinámico del sistema y el diseño del controlador de velocidad se ha utilizado Simulink, junto con la librería Simscape, ambos integrados en el entorno MATLAB. Estas herramientas permiten modelar sistemas dinámicos de forma visual y analizar su comportamiento temporal antes de llevar cualquier solución al sistema físico.

En el marco de este proyecto, Simulink se ha utilizado para representar el comportamiento dinámico del conjunto motor-ESC y para diseñar y validar un controlador PID de velocidad a nivel externo. A través de la simulación ha sido posible estudiar la respuesta del sistema ante cambios en la consigna, evaluar su estabilidad y ajustar los parámetros del controlador con criterios claros y reproducibles.

Además, Simscape ha facilitado la representación de componentes físicos mediante modelos basados en ecuaciones, lo que ayuda a entender mejor la interacción entre las distintas dinámicas del sistema. En conjunto, el uso de Simulink y Simscape ha proporcionado una base sólida para el diseño del control, reduciendo la necesidad de pruebas directas sobre el hardware y permitiendo interpretar los resultados de forma más clara y estructurada antes de la aplicación práctica.

3.5 Configuración del microcontrolador e integración de micro-ROS

Para integrar el microcontrolador dentro del entorno ROS 2 se ha utilizado la placa de desarrollo STM32 NUCLEO-F446ZE, basada en el microcontrolador STM32F446ZET6. Se eligió esta plataforma porque era totalmente compatible con el trabajo: por un lado, ofrece compatibilidad directa con micro-ROS y, por otro, estaba disponible en el laboratorio y resultaba familiar gracias a experiencias previas con el ecosistema STM32. Todo ello permitió avanzar con fluidez desde el comienzo del proyecto.

El microcontrolador STM32F446ZET6 incorpora un núcleo ARM Cortex-M4 que opera a 180 MHz y dispone de un conjunto amplio de periféricos pensados para aplicaciones embebidas exigentes. Entre ellos destacan los temporizadores avanzados, las interfaces UART/USART, los conversores analógico-digitales y el soporte para sistemas operativos en tiempo real. Estas prestaciones lo hacen muy adecuado para ejecutar un nodo micro-ROS, gestionar la comunicación con ROS 2 y, al mismo tiempo, interactuar con el controlador del motor.

La configuración inicial del microcontrolador se hizo usando STM32CubeMX, integrado dentro de STM32CubeIDE, lo que facilitó enormemente el proceso de puesta en marcha. A partir de la selección del STM32F446ZET6 en la herramienta se procedió a habilitar y parametrizar los periféricos necesarios para el correcto funcionamiento del sistema. Este enfoque permitió generar una base de configuración sólida sobre la que posteriormente se integró micro-ROS.

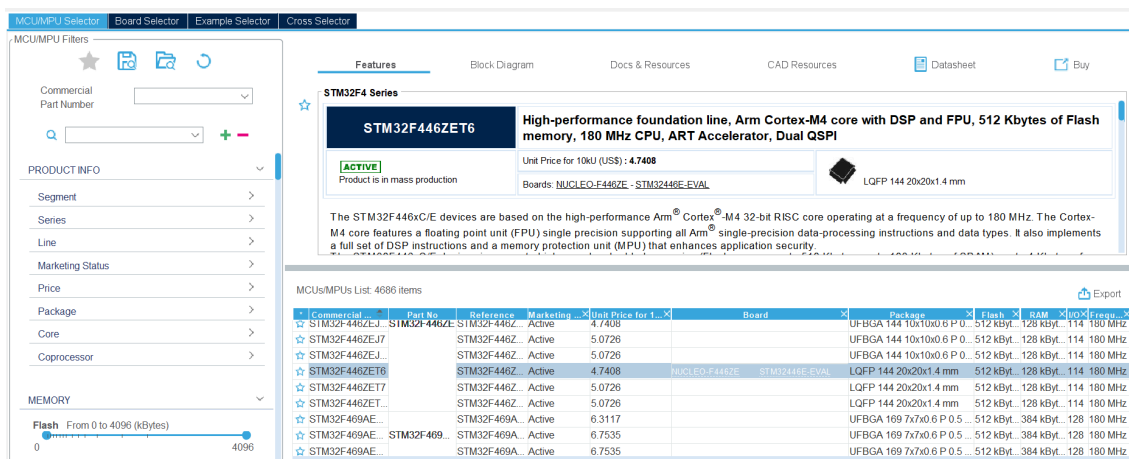


Figura 3.10. Selección del microcontrolador en STM32CubeMX. Fuente: Propia.

3.5.1 Configuración de FreeRTOS

Uno de los primeros pasos durante la configuración del sistema embebido fue la activación de FreeRTOS como sistema operativo en tiempo real. La verdad es que el uso de un RTOS es imprescindible en un sistema como este, donde es necesario coordinar de forma ordenada varias tareas que se ejecutan de manera concurrente, como la recepción de mensajes a través de micro-ROS, el tratamiento de las consignas de velocidad y el envío de comandos al ESC mediante la interfaz UART.

La configuración de FreeRTOS se realizó desde STM32CubeMX, utilizando la interfaz CMSIS_V2, que facilita tanto la portabilidad del código como la integración con librerías externas, entre ellas micro-ROS. En la Figura 3.11 se muestra la configuración adoptada, en la que se habilitan los elementos necesarios para que el sistema operativo funcione de forma estable dentro del entorno embebido del sistema [24].

Gracias a la incorporación de FreeRTOS, el software pudo organizarse de manera mucho más clara y estructurada, separando responsabilidades en tareas independientes y bien definidas.

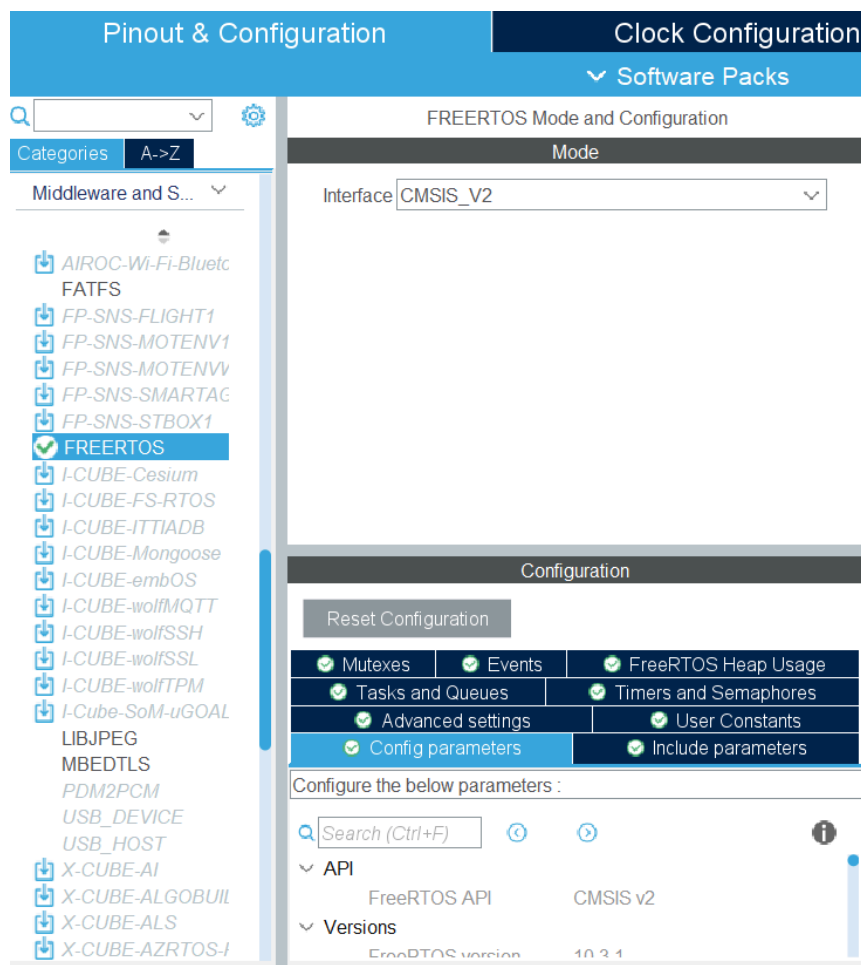


Figura 3.11. Configuración del middleware FreeRTOS. Fuente: Propia.

3.5.2 Configuración de la comunicación USART

La comunicación con el controlador del motor BLDC se realizó a través del periférico USART2, configurado en modo asíncrono. Esta interfaz se utiliza para enviar comandos en formato texto al ESC B-G431B-ESC1, que interpreta dichas tramas como consignas de velocidad a aplicar sobre el motor. Se trata, por tanto, de un enlace directo y dedicado entre el microcontrolador y el sistema de potencia.

La configuración del USART2 se estableció con una velocidad de transmisión de 115200 baudios, sin paridad y con un único bit de parada. Esta configuración, bastante habitual en sistemas embebidos, es suficiente para asegurar una comunicación estable y fiable en el sistema desarrollado. En la Figura 3.12 se muestra la configuración básica del periférico realizada a través de la herramienta STM32CubeMX [16].

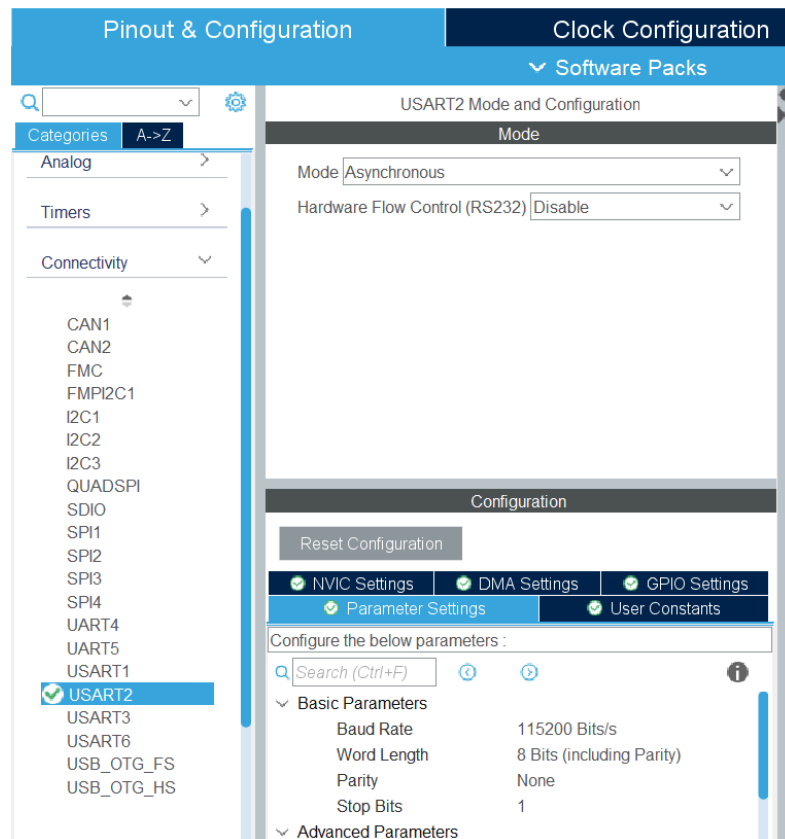


Figura 3.12. Parámetros de configuración del periférico USART2. Fuente: Propia.

3.5.3 Configuración de temporizadores y generación de PWM

Además de la configuración de las interfaces de comunicación, se configuró el temporizador TIM3 con el propósito de disponer de generación de señales PWM directamente desde el microcontrolador. Aunque en el sistema final el control de potencia del motor queda completamente delegado en el ESC, la posibilidad de generar PWM desde el STM32 resultó especialmente útil durante la fase de pruebas iniciales y en tareas de verificación y simulación previas.

En concreto, mediante la herramienta STM32CubeMX se habilitó el canal 1 del temporizador TIM3 en modo PWM Generation CH1. Esta configuración permite generar una señal PWM con ciclo de trabajo ajustable, lo que facilita comprobar el correcto funcionamiento de los temporizadores, validar configuraciones y disponer de una señal de control simple sin necesidad de involucrar aún al ESC [16].

En la *Figura 3.13* se muestra la configuración aplicada al temporizador, donde se definen los parámetros básicos de funcionamiento, como el prescaler, el periodo y el modo de salida. Aunque esta funcionalidad no se utiliza de forma directa en la versión final del sistema, aporta flexibilidad al desarrollo y favorece la comprensión del comportamiento temporal del microcontrolador dentro del conjunto del sistema

embebido.

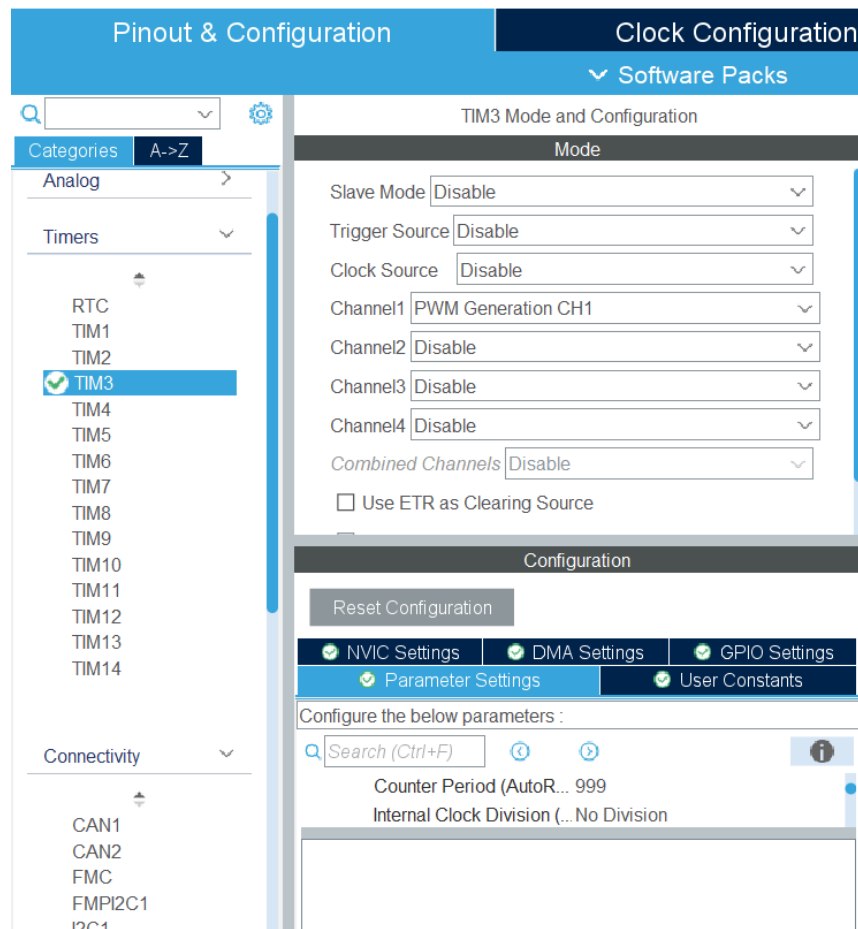


Figura 3.13. Configuración del Timer 3 (TIM3) para generación de PWM. Fuente: Propia.

3.5.4 Integración de micro-ROS en STM32CubeIDE

Una vez configurados todos los periféricos necesarios en STM32CubeMX, se procedió a generar el proyecto base e importarlo en STM32CubeIDE, donde se completó la integración de micro-ROS dentro del sistema embebido. Este proceso se llevó a cabo utilizando la herramienta oficial `micro_ros_stm32cubemx_utils`, que facilita la generación de una librería estática, `libmicroros.a`, adaptada tanto al microcontrolador seleccionado como al sistema operativo en tiempo real empleado.

La librería generada, junto con los archivos asociados, se añadió manualmente al proyecto de STM32CubeIDE. Además, fue necesario configurar de forma explícita las rutas de inclusión y enlace dentro de las opciones de compilación, de modo que el entorno de desarrollo pudiera reconocer correctamente las funciones y estructuras propias de micro-ROS. Este paso fue clave para habilitar la creación y gestión de nodos micro-ROS desde el código embebido.

A continuación, en el código fuente principal del proyecto se incorporaron los *includes* y las llamadas de inicialización necesarias para poner en marcha el nodo micro-ROS. La comunicación entre el microcontrolador y el agente micro-ROS se estableció mediante la función `rmw_uros_set_custom_transport()`, configurando el periférico UART3 como canal de transporte. Esta elección permite una comunicación sencilla y robusta entre el STM32 y el agente ejecutado en el PC.

Gracias a esta configuración, el microcontrolador pudo conectarse correctamente al agente micro-ROS y comportarse como un nodo más dentro del ecosistema ROS 2, suscribiéndose al tópico `/darjan/demanded_speed` y recibiendo las consignas de velocidad generadas en el nivel superior del sistema.

3.5.5 Validación de la integración

La integración de micro-ROS en el sistema embebido permitió que el nodo ejecutado en el STM32 recibiera correctamente las consignas de velocidad publicadas desde ROS 2 y las procesara para su posterior envío al controlador del motor a través de la interfaz UART. Una vez completada la configuración, se realizaron diferentes pruebas funcionales con el objetivo de comprobar que todo el flujo de comunicación funcionaba correcta y sin errores.

Durante estas pruebas se verificó, en primer lugar, que los mensajes publicados en ROS 2 eran recibidos por el nodo micro-ROS de forma correcta. A continuación, se comprobó que las consignas se transformaban correctamente en los comandos UART definidos y que estos eran transmitidos de forma adecuada hacia el ESC. Este proceso permitió confirmar que la lógica de comunicación y conversión de datos estaba correctamente implementada.

Para reforzar esta validación, se empleó un osciloscopio con el fin de observar directamente las tramas UART generadas por el microcontrolador. Gracias a ello, fue posible comprobar el formato de los mensajes, su temporización y la respuesta del sistema ante cambios en la consigna de velocidad. Estas comprobaciones fueron muy útiles para detectar posibles problemas de sincronización o errores de transmisión.

En conjunto, estas pruebas permitieron asegurar que la integración de micro-ROS y la comunicación con el ESC funcionaban según lo esperado. De esta forma se consiguió una base fiable en la que se realizaron diferentes pruebas para confirmar que todo funcionaba correctamente.

3.6 Implementación de un nodo ROS 2 para la publicación

de la consigna de velocidad

Como parte del sistema de comunicación propuesto, se desarrolló un nodo ROS 2 que se ejecuta en el ordenador principal y cuya función es publicar de manera periódica una consigna de velocidad en un tópico determinado. Esta consigna representa el origen del flujo de control del sistema y constituye la referencia que, más adelante, será recibida por el microcontrolador STM32 a través de micro-ROS y enviada por UART al controlador del motor BLDC.

La implementación de este nodo permite separar claramente la generación de la consigna de velocidad del resto del sistema, manteniendo el control de alto nivel en el entorno ROS 2. De este modo, el comportamiento global del sistema se puede modificar fácilmente sin necesidad de alterar el código embebido ni el hardware de control del motor.

Para el desarrollo del nodo se utilizó Python como lenguaje de programación, aprovechando la buena integración que ofrece con ROS 2. Esta elección facilita la creación rápida de nodos y la gestión de la comunicación mediante el modelo publish/subscribe, permitiendo centrarse en la lógica del sistema sin introducir complejidad innecesaria.

3.6.1 Implementación del nodo publicador

El núcleo funcional del paquete se implementó en el script `demanded_speed_node.py`, que define una clase denominada `DemandedSpeedPublisher`, heredada de la clase `Node` proporcionada por la librería `rclpy` de ROS 2.

Este nodo crea un publicador ROS 2 que envía mensajes de tipo `std_msgs/msg/Int32` en el tópico `/darjan/demanded_speed`. La frecuencia de publicación se fijó inicialmente en 1 Hz, suficiente para validar el sistema de comunicación y el correcto funcionamiento del nodo embebido.

La consigna de velocidad utilizada durante las pruebas iniciales fue un valor constante de 1500, representando una velocidad de referencia que será interpretada posteriormente por el sistema embebido y el controlador del motor.

El funcionamiento del nodo se basa en el uso de un temporizador creado mediante el método `create_timer`, que ejecuta periódicamente una función de callback. En cada ejecución de este callback, se genera el mensaje que corresponde y se publica en el tópico utilizando el método `publish()`. El valor enviado se muestra por consola, facilitando la verificación del funcionamiento del nodo.

3.6.2 Configuración para la ejecución del nodo

Para permitir la ejecución del nodo mediante el comando `ros2 run`, fue necesario declarar la función principal del script (`main`) como punto de entrada dentro del archivo `setup.py`. Esta configuración se realizó utilizando la estructura `entry_points` propia del sistema de instalación de ROS 2 para paquetes desarrollados en Python.

Una vez completada esta configuración, el paquete pudo ser correctamente compilado junto con el resto del workspace utilizando la herramienta `colcon`.

3.6.3 Papel del nodo en el sistema global

El papel de este nodo dentro del sistema global es servir como punto de partida de todo el flujo de control. Desde el ordenador host, el nodo se encarga de generar y publicar una consigna de velocidad que será interpretada por el nodo embebido que se ejecuta en el microcontrolador STM32 a través de micro-ROS. Una vez recibida esta consigna, el microcontrolador la procesa y la reenvía mediante comunicación UART al controlador del motor BLDC.

De esta forma, el nodo ROS 2 actúa como el primer bloque de la cadena de control. Su función muestra de forma clara el papel de ROS 2 como capa de alto nivel dentro de una arquitectura distribuida, donde la generación de consignas se mantiene desacoplada del hardware físico encargado del movimiento del motor BLDC.

3.7 Transferencia de señal por UART

Una vez implementado el nodo ROS 2 encargado de publicar la consigna de velocidad en el tópico `/darjan/demanded_speed`, se diseñó un sistema embebido capaz de suscribirse a este tópico, interpretar los datos recibidos y transmitirlos por comunicación serie UART hacia el sistema de control del motor..

3.7.1 Arquitectura de comunicación serie

La arquitectura de comunicación serie implementada en el microcontrolador STM32 se apoya en el uso de dos interfaces UART independientes, cada una con un propósito bien definido dentro del sistema. Esta separación responde a la necesidad de

organizar claramente los distintos flujos de información existentes en el sistema.

Por un lado, la comunicación entre el ordenador y el microcontrolador se lleva a cabo mediante micro-ROS, utilizando el periférico USART3 como canal de transporte. A través de esta interfaz, el nodo embebido puede recibir los mensajes publicados en el entorno ROS 2 gracias a la intermediación del agente micro-ROS, integrándose así de forma transparente en el sistema distribuido.

Por otro lado, la transmisión de la consigna de velocidad hacia el sistema de control se realiza a través del periférico USART2, que actúa como canal de salida de datos. Esta interfaz se emplea para enviar comandos en formato texto al controlador electrónico de velocidad (ESC).

El uso de dos interfaces diferenciadas permite desacoplar la comunicación asociada a micro-ROS del envío de comandos de control, lo que simplifica el diseño del software embebido y implica una mayor claridad de la arquitectura global de sistema.

3.7.2 Suscripción al tópico y procesamiento de la consigna

El microcontrolador ejecuta un nodo micro-ROS que se suscribe al tópico `/darjan/demanded_speed`, recibiendo mensajes de tipo `std_msgs/msg/Int32`. Cada vez que se recibe un nuevo mensaje, se ejecuta una función de callback asociada a la suscripción.

En esta función, el valor recibido se procesa a nivel de aplicación y se convierte en una cadena de texto con un cierto formato específico, diseñado para ser interpretado fácilmente por el sistema de control del motor. El formato definido es:

`SPD<valor>\n`

De este modo, por ejemplo, una consigna de velocidad con valor 1500 se convierte en el mensaje:

`SPD1500\n`

Este formato permite identificar de forma clara el tipo de comando enviado y el valor asociado, manteniendo la comunicación simple y extensible.

3.7.3 Implementación de la transmisión UART

La conversión de la consigna recibida y su posterior transmisión por UART se realiza

dentro de la función de callback del nodo micro-ROS. El fragmento principal de código responsable de esta conversión i transmisión es el siguiente:

```
void subscription_callback(const void * msgin)
{
    const std_msgs__msg__Int32 * msg = (const std_msgs__msg__Int32 *)msgin;
    snprintf(uart_tx_buffer, sizeof(uart_tx_buffer), "SPD%d\n", (long int)msg->data);
    if (uart_tx_ready)
    {
        uart_tx_ready = 0;
        HAL_UART_Transmit_IT(&huart2, (uint8_t *)uart_tx_buffer,
        strlen(uart_tx_buffer));
    }
    HAL_GPIO_TogglePin(GPIOB, LD2_Pin); // LED de debug
}
```

En este código, el valor recibido desde ROS 2 se formatea usando la función `snprintf` y se almacena en un buffer de transmisión. La comunicación por UART se realiza utilizando el modo de transmisión mediante interrupciones (`HAL_UART_Transmit_IT()`), lo que permite una transmisión que no sea bloqueante.

Para evitar conflictos durante la escritura en el periférico UART, se emplea un indicador de estado, en este caso el flag (`uart_tx_ready`), que permite conocer en todo momento si el canal de transmisión está disponible. De esta forma, se garantiza que no se lance una nueva transmisión hasta que la anterior haya finalizado correctamente, evitando solapamientos y posibles errores en el envío de datos.

3.7.4 Configuración de los periféricos UART

Para asegurar que la transmisión de datos se realizara de forma fiable, los periféricos UART del microcontrolador se configuraron utilizando las herramientas STM32CubeMX y STM32CubeIDE, siguiendo parámetros de comunicación estándar y contrastados en sistemas embebidos.

En concreto, se emplearon dos interfaces serie con funciones bien diferenciadas dentro del sistema. Por un lado, el periférico USART3 se destinó a la comunicación entre el microcontrolador y el agente micro-ROS, actuando como canal de enlace con el entorno ROS 2. Por otro, el periférico USART2 se utilizó exclusivamente para el envío de comandos hacia el sistema de control del motor, ya fuera el ESC físico o un entorno de simulación. Ambos periféricos se configuraron en modo asíncrono, con una velocidad de transmisión de 115200 baudios, 8 bits de datos, sin paridad y un bit de parada [16].

3.7.5 Papel de la comunicación UART en el sistema

La comunicación UART representa un papel muy importante dentro del sistema desarrollado, ya que actúa como el último bloque de la cadena que conecta el entorno distribuido basado en ROS 2 con el controlador físico del motor. A través de este enlace, las consignas de velocidad generadas en el nivel de alto nivel se transforman en comandos simples y directamente interpretables por el sistema de control de potencia.

Este planteamiento permite separar de forma clara la lógica de comunicación y supervisión, implementada en el microcontrolador, del control eléctrico propiamente dicho del motor, que queda encapsulado en el ESC.

La correcta integración y validación de este mecanismo de transferencia por UART, así como su funcionamiento coordinado con el resto de los bloques del sistema, se analizan con mayor detalle en el capítulo de resultados.

3.8 Recepción de la consigna de velocidad por UART y aplicación al motor

Una vez que la consigna de velocidad ha sido enviada desde el microcontrolador principal a través de la interfaz UART, el siguiente paso del proceso consiste en su correcta recepción e interpretación por parte del sistema encargado del control del

motor. En el sistema desarrollado, esta función recae en el controlador del motor BLDC, que recibe los comandos serie, extrae el valor de velocidad deseada y lo aplica al motor utilizando las funciones de control disponibles en su API.

En este apartado se describe cómo se ha implementado la recepción de los comandos UART, el tratamiento del mensaje recibido y su integración con el control de velocidad del motor, prestando especial atención a la robustez y simplicidad del mecanismo utilizado.

3.8.1 Estrategia de recepción y gestión del buffer

La recepción de datos por UART se ha implementado siguiendo una estrategia sencilla pero eficaz, basada en la lectura secuencial de los bytes que llegan por el canal serie. Cada byte recibido se almacena en un buffer de tamaño fijo, utilizando un índice que avanza progresivamente a medida que se reciben nuevos datos.

La lectura se realiza dentro del bucle principal del programa mediante la función `HAL_UART_Receive()`, configurada para recibir un único byte en cada llamada. Este planteamiento permite un control muy preciso del flujo de datos y facilita la detección del final de cada mensaje sin depender de longitudes predefinidas ni de temporizaciones externas, lo que aporta flexibilidad al sistema.

El carácter de salto de línea (`\n`) se emplea como delimitador de fin de mensaje. Cuando se detecta este carácter, se considera que el comando ha sido recibido por completo y se procede a su procesamiento. A continuación, el índice del buffer se reinicia, quedando el sistema preparado para la recepción del siguiente comando.

Para garantizar un funcionamiento robusto, se incluye además una comprobación del tamaño del buffer. Si el índice alcanza el límite máximo permitido, este se reinicia automáticamente, evitando así posibles desbordamientos de memoria y asegurando la estabilidad del sistema incluso ante errores de transmisión.

3.8.2 Procesamiento del mensaje y extracción de la velocidad

Una vez recibido el mensaje completo, el contenido del buffer se trata como una cadena de texto. Primero, se comprueba que el mensaje comienza con el identificador "SPD", lo que permite verificar que se trata de una consigna de velocidad válida enviada desde el sistema de control.

Si el formato es válido, se extrae la parte numérica del mensaje, que corresponde a la

velocidad deseada del motor. Esta conversión se realiza usando funciones estándar de manejo de cadenas, transformando el valor textual en un entero que puede ser utilizado directamente por el sistema de control.

El siguiente fragmento de código muestra la lógica empleada para la recepción y el procesamiento del mensaje UART:

```
char rx_buffer[16];

int index = 0;

while (1)
{
    uint8_t byte;

    HAL_UART_Receive(&huart2, &byte, 1, HAL_MAX_DELAY);

    if (index < (int)(sizeof(rx_buffer) - 1))
    {
        rx_buffer[index++] = byte;

        if (byte == '\n') // Fin de mensaje
        {
            rx_buffer[index] = '\0'; // Cierre del string

            if (strncmp(rx_buffer, "SPD", 3) == 0)
            {
```

```
int velocidad_deseada = atoi(&rx_buffer[3]);

// La velocidad ya está disponible para el control del motor

}

index = 0; // Preparado para el siguiente mensaje

}

else

{

    // Protección frente a overflow

    index = 0;

}

}
```

Este método permite implementar una recepción sencilla, clara y fácilmente extensible, adecuada para sistemas embebidos con comunicación serie asíncrona.

3.8.3 Aplicación de la consigna al control del motor BLDC

Una vez extraído el valor de velocidad deseada a partir del mensaje recibido por UART, esta información se utiliza directamente como referencia para el control del motor BLDC. En el sistema empleado, el controlador del motor proporciona una API de alto nivel que simplifica notablemente esta tarea, permitiendo tanto el arranque del motor como la programación de rampas de velocidad de manera segura y controlada.

En concreto, cuando se recibe una consigna válida, el sistema inicia el funcionamiento del motor y establece una rampa de aceleración hasta alcanzar la velocidad objetivo.. El tiempo de la rampa puede ajustarse según las necesidades de la aplicación,

permitiendo priorizar una respuesta rápida o, un arranque más progresivo.

De este modo, la consigna recibida desde el sistema distribuido basado en ROS 2 se traduce en una acción física sobre el motor de forma gradual y controlada, cerrando el ciclo de comunicación y control. El siguiente fragmento de código ilustra cómo se integra la velocidad recibida con las funciones de control del motor proporcionadas por la API del ESC.

```
if (strncmp(rx_buffer, "SPD", 3) == 0)
{
    int velocidad_deseada = atoi(&rx_buffer[3]);

    MC_StartMotor1();

    MC_ProgramSpeedRampMotor1(velocidad_deseada, 5000);
}
```

En este caso, se programa una rampa de velocidad de 5000 ms, permitiendo que el motor alcance progresivamente la velocidad solicitada.

3.8.4 Integración dentro del sistema distribuido

La recepción de la consigna por UART y su aplicación al motor forman parte de el último paso de la arquitectura distribuida desarrollada en este trabajo. A través de este mecanismo, una consigna generada en el entorno ROS 2 es finalmente transformada en una acción física sobre el motor BLDC.

El sistema completo se implementó y probó de manera integrada, recorriendo todo el flujo de control: desde la publicación del mensaje en ROS 2, pasando por su transmisión mediante micro-ROS y la comunicación serie, hasta la actuación del ESC sobre el motor. Aunque no fue posible llevar a cabo una validación experimental exhaustiva de cada etapa por separado debido a las limitaciones temporales del

proyecto, las pruebas realizadas permitieron verificar que el motor respondía de forma coherente a las consignas de velocidad recibidas.

Este resultado confirma que la arquitectura propuesta funciona de manera consistente y que la integración entre ROS 2, micro-ROS, el sistema embebido y el controlador de potencia es correcta. En conjunto, el comportamiento observado respalda el enfoque distribuido que se ha utilizado en este trabajo y demuestra la viabilidad del sistema desarrollado para el control de velocidad de un motor BLDC en un entorno real.

4. Resultados y discusión

4.1 Validación del sistema de comunicación ROS 2 – micro-ROS

En este capítulo se muestran los resultados obtenidos durante la validación del sistema de comunicación realizado. El análisis se plantea de manera ordenada, comenzando por la verificación del entorno ROS 2 ejecutado en el ordenador y avanzando hasta su integración con el sistema embebido y el hardware encargado del control del motor.

En un inicio, se comprueba la correcta publicación de la consigna de velocidad desde el nodo ROS 2, sin involucrar todavía al microcontrolador ni al sistema físico. De esta forma, se puede aislar el funcionamiento del software de alto nivel y asegurarse de que la información generada es correcta antes de introducir más elementos en la arquitectura.

4.1.1 Ejecución del agente micro-ROS

El primer paso en la validación de la comunicación entre ROS 2 y el microcontrolador STM32 consistió en verificar la correcta ejecución del agente micro-ROS en el ordenador host. Este agente es una pieza clave dentro de la arquitectura del sistema, ya que actúa como puente entre los nodos ROS 2 convencionales y los nodos embebidos que se ejecutan en microcontroladores como el utilizado en este trabajo.

El agente micro-ROS se ejecuta como un proceso independiente en el ordenador y es el encargado de traducir los mensajes entre el middleware DDS utilizado por ROS 2 y el protocolo XRCE-DDS utilizado por micro-ROS. Gracias a este proceso, el nodo embebido puede integrarse en el ecosistema ROS 2 sin necesidad de implementar directamente la pila DDS completa en el microcontrolador [6].

Durante las pruebas realizadas, el agente micro-ROS se lanzó correctamente desde la terminal y permaneció en ejecución, a la espera de conexiones entrantes procedentes del microcontrolador. Este comportamiento confirma que el entorno software necesario para la comunicación entre ROS 2 y micro-ROS está configurado de forma correcta.

En la *Figura 4.1* se muestra una captura del agente micro-ROS ejecutándose en el ordenador host, donde se puede apreciar que el proceso se encuentra activo y preparado para gestionar la comunicación con el nodo embebido.

```

ros2-student@ros2z400: ~/microros_ws
[1751306526.501474] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 12 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.503480] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 1A 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.505390] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 13 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.507461] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 1B 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.508483] debug | SerialAgentLinux.cpp | recv_message | [==>> SER <==>] | client_key: 0x5851F42D, len: 13, data:
0000: 81 00 00 00 0A 01 05 00 1A 00 00 3F 80
[1751306526.509401] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 14 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.510450] debug | SerialAgentLinux.cpp | recv_message | [==>> SER <==>] | client_key: 0x5851F42D, len: 13, data:
0000: 81 00 00 00 0A 01 05 00 1B 00 00 1F 80
[1751306526.511445] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 1C 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.513405] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 15 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.515443] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 1D 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.517431] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 16 00 09 01 08 00 00 0E 00 06 DC 05 00 00
[1751306526.519451] debug | SerialAgentLinux.cpp | send_message | [** <<SER>> **] | client_key: 0x5851F42D, len: 16, data:
0000: 81 80 17 00 09 01 08 00 00 0E 00 06 DC 05 00 00

```

Figura 4.1. Agente micro-ROS activo. Fuente: Propia.

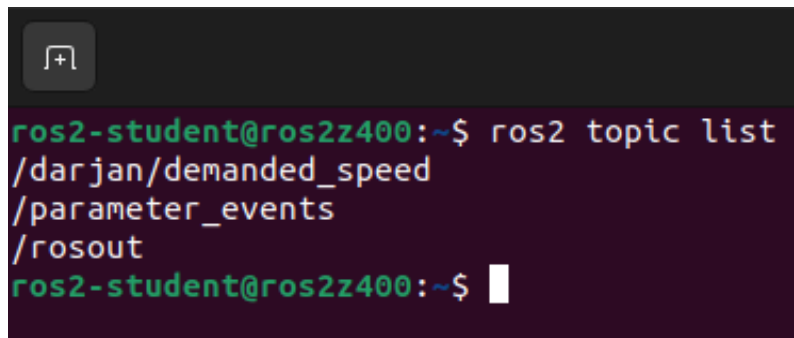
4.1.2 Verificación del grafo de comunicación en ROS 2

Una vez comprobado que el agente micro-ROS se encontraba en ejecución y funcionando correctamente, el siguiente paso era verificar el estado del grafo de comunicación en ROS 2. Esta comprobación es realmente importante para asegurarse de que los nodos y tópicos del sistema están correctamente registrados y de que la arquitectura distribuida se comporta tal y como se ha diseñado y previsto.

Para ello, se recurrió a las herramientas de inspección de ROS 2, en particular a los comandos `ros2 topic list` y `ros2 topic info`. Mediante la ejecución de `ros2 topic list` se pudo confirmar la existencia del tópico `/darjan/demanded_speed`, lo que indica que el nodo publicador encargado de generar la consigna de velocidad estaba activo y correctamente integrado en el entorno ROS 2.

A continuación, utilizando el comando `ros2 topic info /darjan/demanded_speed`, se analizó con más detalle la información asociada a dicho tópico. En esta salida se observó la presencia de un publicador y varios suscriptores, lo que pone de manifiesto que el sistema está formado por múltiples nodos que intercambian información de manera distribuida. Entre estos suscriptores aparece el nodo embebido basado en micro-ROS, lo que confirma que el microcontrolador se integra en el grafo de comunicación de ROS 2 como un nodo más, sin necesidad de mecanismos de comunicación adicionales.

En la *Figura 4.2* se muestra una captura representativa de la salida de estos comandos, donde se puede ver tanto la correcta creación del tópico como la relación entre publicadores y suscriptores. Esta verificación permite concluir que la infraestructura de comunicación de ROS 2 funciona forma correcta y que el sistema distribuido esta respondiendo de forma esperada antes de avanzar hacia la validación de la transferencia de datos y su integración con el hardware físico.



```
ros2-student@ros2z400:~$ ros2 topic list
/darjan/demanded_speed
/parameter_events
/rosout
ros2-student@ros2z400:~$
```

Figura 4.2. Verificación tópico. Fuente: Propia.

```
ros2-student@ros2z400:~$ ros2 topic info /darjan/demanded_speed

Type: std_msgs/msg/Int32

Publisher count: 1

Subscription count: 2
```

Código Verificación tópico y suscripción al mismo. Fuente: Propia.

4.1.3 Verificación de la publicación de la consigna de velocidad

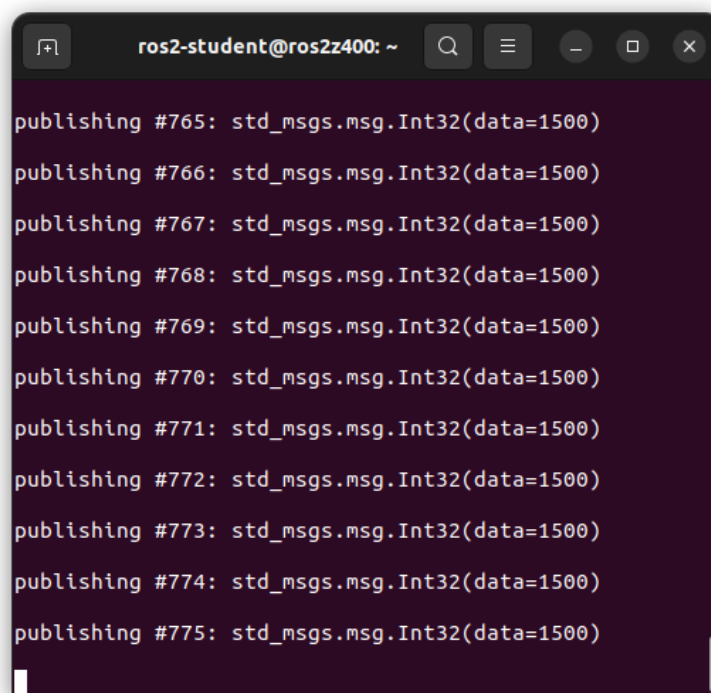
La primera etapa de la validación del sistema se centró en comprobar que el nodo ROS 2 desarrollado es capaz de publicar correctamente la consigna de velocidad en el tópico generado: `/darjan/demanded_speed`.

Mediante el comando `ros2 topic echo /darjan/demanded_speed` se pudo observar, directamente desde la terminal, la publicación periódica de mensajes en el tópico creado. Los valores mostrados coincidían con la consigna configurada en el nodo, confirmando por tanto, que el publicador genera y transmite correctamente los datos esperados.

Además, se verificó que la frecuencia de publicación concuerda con la definida durante la implementación del nodo. En las pruebas realizadas, la frecuencia fue de 1 Hz, un valor adecuado para el envío de consignas de velocidad en el contexto del sistema desarrollado.

Por otro lado, también se comprobó el tipo de mensaje utilizado. Los datos publicados corresponden al tipo `std_msgs/msg/Int32`, tal y como se había especificado durante el desarrollo. Este tipo de mensaje es muy válido para representar consignas de velocidad y garantiza la compatibilidad con el nodo micro-ROS encargado de recibir la información en el sistema embebido.

En la *Figura 4.3* se muestra una captura representativa de la salida del comando `ros2 topic echo /darjan/demanded_speed`, donde puede observarse la publicación periódica de la consigna de velocidad con el valor esperado. Esta verificación confirma el correcto funcionamiento del nodo ROS 2 y valida la primera etapa del sistema de comunicación desarrollado en el trabajo.

A terminal window titled 'ros2-student@ros2z400: ~' displays the output of the command 'ros2 topic echo /darjan/demanded_speed'. The output shows a series of messages from #765 to #775, each containing 'std_msgs.msg.Int32(data=1500)'. The terminal has a dark background with light-colored text. The window includes standard Linux window controls (minimize, maximize, close) and a search icon in the title bar.

```
ros2-student@ros2z400: ~
publishing #765: std_msgs.msg.Int32(data=1500)
publishing #766: std_msgs.msg.Int32(data=1500)
publishing #767: std_msgs.msg.Int32(data=1500)
publishing #768: std_msgs.msg.Int32(data=1500)
publishing #769: std_msgs.msg.Int32(data=1500)
publishing #770: std_msgs.msg.Int32(data=1500)
publishing #771: std_msgs.msg.Int32(data=1500)
publishing #772: std_msgs.msg.Int32(data=1500)
publishing #773: std_msgs.msg.Int32(data=1500)
publishing #774: std_msgs.msg.Int32(data=1500)
publishing #775: std_msgs.msg.Int32(data=1500)
```

Figura 4.3. Publicación periódica de la consigna de velocidad. Fuente: Propia.

4.2 Resultados de la transferencia de señal por UART

Una vez comprobado que la consigna de velocidad se publica correctamente dentro del entorno ROS 2, el siguiente paso consistió en analizar el comportamiento del sistema durante la transferencia de la señal a través de la comunicación serie UART.

Esta etapa es muy importante, ya que representa el punto de unión entre el sistema distribuido de alto nivel basado en ROS 2 y el subsistema encargado del control físico del motor BLDC.

En este apartado se muestran los resultados obtenidos durante la transmisión de la consigna desde el microcontrolador STM32, su recepción, e interpretación por parte del controlador del motor. El objetivo es verificar que la información generada en el entorno software llega de forma correcta y esperada al sistema de potencia, asegurando que el flujo de datos se ha dado con éxito desde ROS 2 hasta el actuador final.

4.2.1 Transmisión de la consigna desde el STM32 por UART

Una vez que el nodo micro-ROS recibe la consigna de velocidad procedente del entorno ROS 2, el microcontrolador STM32 se encarga de procesar este valor y prepararlo para su envío al controlador del motor. Para ello, la consigna se formatea y se transmite a través del periférico UART2 en forma de comando de texto, siguiendo la siguiente estructura:

`SPD<valor>\n`

donde `<valor>` representa la velocidad deseada del motor expresada como un número entero. Este formato, permite identificar de forma inmediata el tipo de comando enviado y facilita la interpretación por parte del sistema receptor.

Durante las pruebas realizadas se pudo comprobar que, cada vez que se publicaba una nueva consigna desde ROS 2, el STM32 generaba y transmitía el mensaje UART sin errores. El uso de transmisión no bloqueante, basada en interrupciones, fue muy efectiva para asegurar que el envío de datos no interfería con el resto de tareas del sistema embebido, manteniendo de esta manera, un funcionamiento fluido.

En la *Figura 4.4* se presenta una captura representativa de la salida por UART, en la que se puede observar la correcta transmisión de los comandos con el formato definido. La coincidencia entre el valor publicado en ROS 2 y el mensaje enviado por el STM32 confirma que la cadena de comunicación funciona correctamente hasta este punto del sistema.

Expression	Type	Value
> uart2	UART_HandleTypeDef	{...}
▼ uart_tx_buffer	char [32]	0x2000083c <uart_tx
uart_tx_buffer[0]	char	83 'S'
uart_tx_buffer[1]	char	80 'P'
uart_tx_buffer[2]	char	68 'D'
uart_tx_buffer[3]	char	49 '1'
uart_tx_buffer[4]	char	53 '5'
uart_tx_buffer[5]	char	48 '0'
uart_tx_buffer[6]	char	48 '0'
uart_tx_buffer[7]	char	10 '\n'
uart_tx_buffer[8]	char	0 '\0'
uart_tx_buffer[9]	char	0 '\0'
uart_tx_buffer[10]	char	0 '\0'
uart_tx_buffer[11]	char	0 '\0'
uart_tx_buffer[12]	char	0 '\0'

Figura 4.4. Transmisión UART en STM32CubeIDE. Fuente: Propia.

4.2.2 Recepción e interpretación del comando por el controlador del motor

El controlador del motor BLDC recibe los comandos enviados por el microcontrolador a través de la interfaz UART y los analiza para extraer la consigna de velocidad que corresponde. Cuando se detecta un mensaje válido que comienza con el prefijo "SPD", el controlador interpreta el valor numérico como la velocidad objetivo que debe ser alcanzada por el motor.

En el sistema empleado, esta consigna se aplica utilizando las funciones proporcionadas por la API del propio controlador. Estas funciones permiten iniciar el funcionamiento del motor y definir rampas de aceleración hasta la velocidad deseada, de manera que la transición no sea brusca.

Durante las pruebas experimentales, el motor respondió correctamente a las consignas transmitidas, variando su velocidad de acuerdo con los valores enviados desde el entorno de ROS 2. La respuesta observada confirma que los comandos UART son recibidos e interpretados correctamente por el controlador del motor y que la cadena de comunicación diseñada funciona de extremo a extremo, desde el software de alto nivel hasta la actuación física sobre el motor.

En la Figura 4.5 se muestra el sistema completo durante una de las pruebas, donde pueden observarse la fuente de alimentación, el microcontrolador STM32, el controlador electrónico de velocidad (ESC) y el motor BLDC, conectados y en funcionamiento.

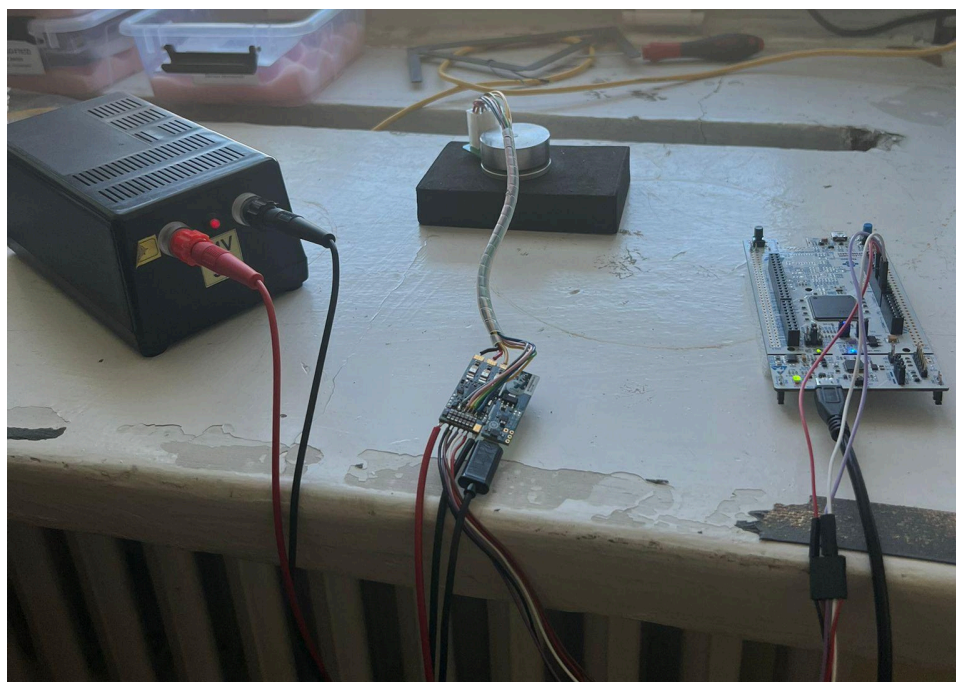


Figura 4.5. Montaje experimental completo con electrónica y motor conectados. Fuente: Propia.

4.3 Simulación del sistema de control de velocidad del motor BLDC

Una vez comprobado que el sistema de comunicación entre ROS 2, micro-ROS y el microcontrolador STM32 funciona de manera correcta, el siguiente paso fue centrarse en el análisis del comportamiento dinámico del sistema a través de la simulación. En esta fase, el objetivo no es reproducir de forma precisa todos los fenómenos eléctricos internos del motor BLDC ni del ESC, sino disponer de un modelo representativo que permita estudiar la respuesta temporal del sistema y, a partir de ella, diseñar un controlador de velocidad adecuado y bien justificado.

Tal y como se ha expuesto en el apartado teórico, el sistema físico utilizado en este trabajo presenta una estructura de control jerárquica. El ESC B-G431B-ESC1 se encarga internamente de los lazos rápidos asociados a la conmutación del motor y al control de corriente, ocultando toda la complejidad eléctrica del accionamiento. El sistema desarrollado en este trabajo, en cambio, actúa a un nivel superior, generando consignas de velocidad que el ESC interpreta y ejecuta.

Desde esta perspectiva, y pensando únicamente en el diseño del control externo de velocidad, el conjunto formado por el motor BLDC y el ESC pueden ser tratados como un único bloque dinámico. Esta simplificación permite aproximar su comportamiento a través de un modelo de primer orden, adecuado para analizar la dinámica dominante del sistema y para realizar el diseño del controlador sin introducir una complejidad innecesaria.

4.3.1 Modelo utilizado en la simulación

Para poder diseñar el controlador de velocidad sin poner en riesgo el hardware real, fue necesario obtener primero un modelo matemático que representase la dinámica del conjunto. Dado que en este proyecto el control de bajo nivel (conmutación y corrientes) lo gestiona internamente el ESC, desde el punto de vista del sistema de control externo el comportamiento es análogo al de un motor de corriente continua (DC) controlado por tensión.

Por ello, se decidió modelar la planta (ESC + Motor) como un sistema de primer orden. Para identificar los parámetros, se realizó un ensayo en lazo abierto aplicando un escalón de entrada y registrando la evolución de la velocidad.

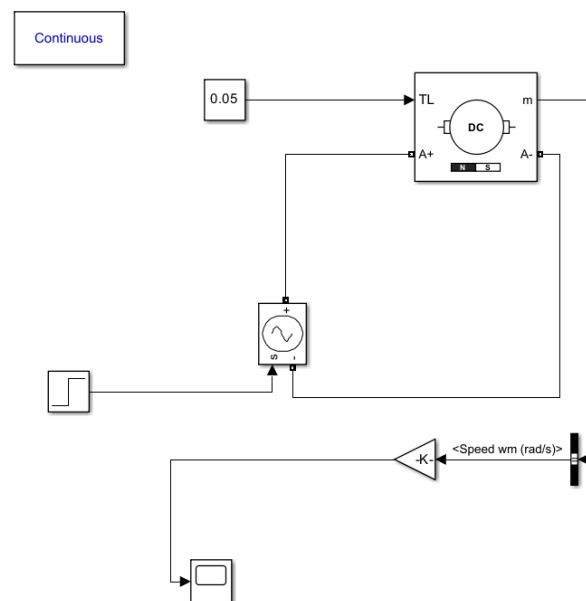


Figura 4.6. Modelo de simulación de la planta en lazo abierto (Simulink). Fuente: Propia.

Tal y como se puede observar en la gráfica de la Figura 4.7, la respuesta del sistema es exponencial y monótona. A partir de los datos visuales de dicha gráfica, se

extrajeron los parámetros característicos de la función de transferencia de primer orden $G_p(s)$:

$$G_p(s) = \frac{K}{1 + \tau * s} \quad \text{Ecuación (4.1)}$$

Donde:

- Ganancia estática (K): Calculada como el cociente entre el incremento de la velocidad final y el escalón de entrada aplicado.

$$K = \frac{15000}{20} = 750 \quad \text{Ecuación (4.2)}$$

- Constante de tiempo (τ): Correspondiente al tiempo que tarda la respuesta en alcanzar el 63,3% de su valor final. Observando la curva, este punto se sitúa en:

$$\tau = 0,04 \text{ s}$$

Sustituyendo estos valores, el modelo de nuestra planta queda definido como:

$$G_p(s) = \frac{750}{1 + 0,04 * s} \quad \text{Ecuación (4.3)}$$

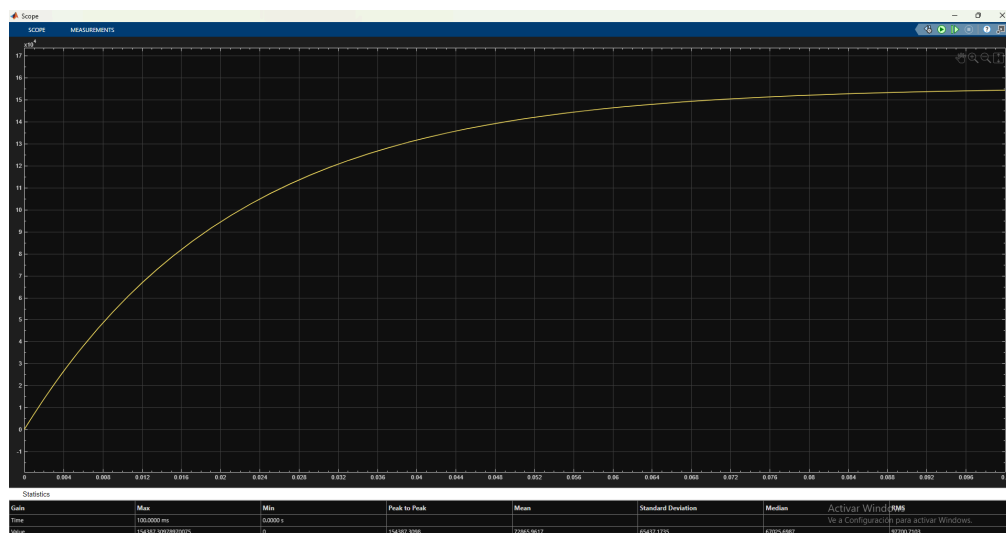


Figura 4.7. Respuesta temporal experimental para identificación del sistema. Fuente: Propia.

4.3.2 Implementación del modelo en Simulink

Una vez identificada la planta, se implementó el esquema de control en Simulink utilizando la librería Simscape. Se empleó un bloque de "DC Machine" para representar la física del motor, alimentado por una fuente controlada que emula la

acción del ESC. El esquema permite cerrar el lazo de velocidad y comparar la respuesta frente a la referencia deseada, tal y como se muestra en el diagrama de la *Figura 4.8*.

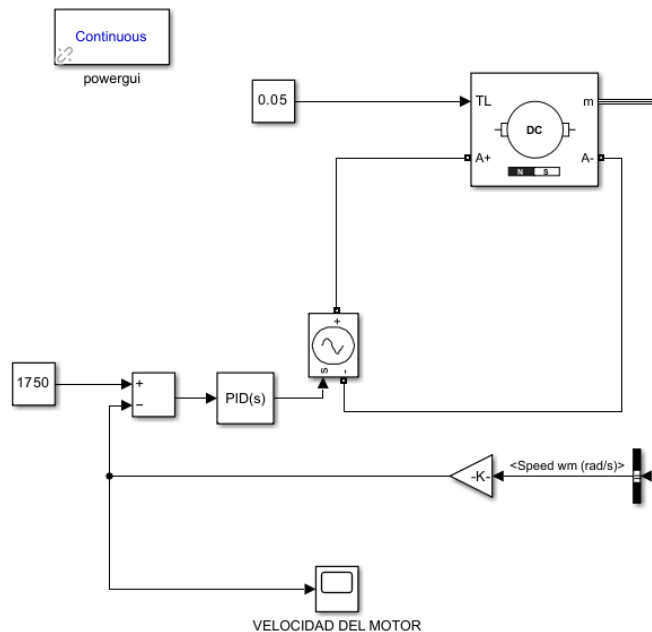


Figura 4.8. Diagrama de bloques del control PID de velocidad en lazo cerrado. Fuente: Propia.

4.3.3 Ajuste del controlador PID

Para el cálculo de los parámetros del controlador se ha utilizado el Método de Síntesis Directa (también conocido como cancelación de polos). Este método analítico permite deducir las ganancias necesarias para que el sistema en lazo cerrado se comporte con una dinámica predefinida por el diseñador.

Partimos de la función de transferencia de nuestra planta (modelo de primer orden):

$$G_p(s) = \frac{750}{1 + 0,04 * s} \quad \text{Ecuación (4.3)}$$

Y consideramos un controlador PI ideal cuya función de transferencia es:

$$G_{pi}(s) = K_p * \left(1 + \frac{1}{T_i * s} \right) = K_p * \frac{1 + T_i * s}{T_i * s} \quad \text{Ecuación (4.4)}$$

El objetivo del método es simplificar la dinámica del sistema haciendo que el tiempo integral del controlador (T_i) iguale a la constante de tiempo de la planta (τ). Es decir, imponemos la condición de diseño:

$$T_i = \tau = 0,04 \text{ s} \quad \text{Ecuación (4.5)}$$

Al realizar esta igualación, el cero del controlador cancela algebraicamente el polo de la planta. Si analizamos la función de transferencia en lazo cerrado $G_c(s)$ del sistema resultante:

$$G_c(s) = \frac{G_{pi}(s) * G_p(s)}{1 + G_{pi}(s) * G_p(s)} \quad \text{Ecuación (4.6)}$$

Sustituyendo y simplificando gracias a la cancelación del término $(1+\tau*s)$ obtenemos una expresión reducida que depende únicamente de las ganancias:

$$G_c(s) = \frac{1}{1 + (\frac{\tau}{K * K_p})} \quad \text{Ecuación (4.7)}$$

Por otro lado, deseamos que nuestro sistema en lazo cerrado se comporte como un sistema de primer orden ideal con una constante de tiempo deseada τ_c :

$$G_{deseada}(s) = \frac{1}{1 + \tau_c * s} \quad \text{Ecuación (4.8)}$$

Igualando los denominadores de ambas expresiones (la obtenida y la deseada), podemos despejar directamente el valor de la ganancia proporcional K_p :

$$(\frac{\tau}{K * K_p}) = \tau_c \quad \text{Ecuación (4.9)}$$

Por tanto:

$$K_p = \frac{\tau}{K * \tau_c}$$

Para el diseño, se ha establecido como requisito que el sistema en lazo cerrado sea el doble de rápido que en lazo abierto, fijando $\tau_c = \tau/2 = 0,02\text{s}$. Sustituyendo los valores de nuestro sistema:

$$K_p = \frac{0,04}{750 * 0,02} = 0,00267 \quad \text{Ecuación (4.10)}$$

Finalmente, para la implementación en el bloque PID paralelo de Simulink, la ganancia integral K_i se obtiene a partir del tiempo integral ($T_i = \tau$) mediante la relación $K_i = K_p / T_i$:

$$K_i = \frac{0,00267}{0,04} = 0,0667 \quad \text{Ecuación (4.11)}$$

De este modo, los parámetros quedan justificados analíticamente:

$$K_p = 0,00267$$

$$K_i = 0,0667$$

4.3.4 Resultados de la simulación en lazo cerrado

Con el controlador ajustado según los cálculos anteriores, se procedió a simular la respuesta del sistema ante una consigna de velocidad de 1750 rpm. En la *Figura 4.9* se muestra la evolución temporal de la velocidad obtenida.

Se puede apreciar que el sistema sigue la referencia correctamente, con una subida suave y sin presentar sobreimpulso, tal y como predice la teoría de cancelación de polos (que busca una respuesta de primer orden pura). La acción integral cumple su función eliminando el error en régimen permanente, logrando que la velocidad final del motor coincida con la solicitada. La dinámica observada confirma la validez del modelo matemático y del método empleado.



Figura 4.9. Respuesta simulada del sistema con el controlador ajustado. Fuente: Propia.

4.3.5 Propuesta de implementación futura en el sistema físico

Aunque en el alcance actual de este proyecto el control de bajo nivel se ha delegado en el hardware del ESC para simplificar la arquitectura, la simulación realizada y el controlador diseñado abren la puerta a una futura implementación del lazo de velocidad directamente en el microcontrolador STM32. Esta evolución permitiría tener un control total sobre la dinámica del sistema, sin depender de los algoritmos internos del ESC, y validar experimentalmente los cálculos realizados a nivel teórico.

Para trasladar la estructura de control simulada en Simulink al sistema físico real, se propone el siguiente esquema de implementación dividido en tres etapas:

1. Reconfiguración del Actuador

En la configuración actual, el ESC gestiona la velocidad de forma autónoma mediante sus propios lazos internos. Para que el microcontrolador principal pueda ejecutar el algoritmo de control diseñado, sería necesario configurar el ESC en modo "Ciclo de Trabajo" (Duty Cycle). De esta manera, el ESC pasaría a actuar como una etapa de potencia, anulando su lógica de control interna y limitándose a generar una señal PWM proporcional a la orden recibida desde el STM32.

2. Instrumentación y Realimentación (Feedback)

Para cerrar el lazo de control, se procedería a la instalación de un encoder magnético en el eje del motor. El hardware del ESC sería el encargado de leer la velocidad real del motor a través de este sensor y enviarla periódicamente al STM32 aprovechando la trama de UART ya existente. Esta mejora en la instrumentación permitiría al microcontrolador conocer el estado del sistema con cierta precisión.

3. Ejecución del PID y Comunicación de la Acción de Control

Bajo este nuevo esquema, el núcleo del control residiría enteramente en el STM32, que ejecutaría el siguiente ciclo en tiempo real:

- Cálculo: El microcontrolador compararía la velocidad real (recibida por UART) con la consigna proveniente de ROS 2. Sobre este error aplicaría el algoritmo PID discretizado para obtener la acción de control $u[k]$, que representa el porcentaje de tensión necesario para alcanzar el valor de velocidad deseado.
- Transmisión por UART: Este valor calculado se enviaría al ESC mediante UART, pero cambiando el tipo de comunicación. En lugar de enviar una consigna de velocidad (como se hace actualmente), el STM32 enviaría un comando de ciclo de trabajo. Se propone un formato del tipo **DTY<valor>**, donde **<valor>** representa el duty cycle deseado (por ejemplo, **DTY450** para solicitar un 45,0% de potencia).
- Actuación: Al recibir el comando **DTY**, el ESC aplicaría ese porcentaje de tensión al motor, cerrando así el bucle de control físico.

Esta arquitectura validaría el diseño realizado en este trabajo, demostrando que el microcontrolador es capaz no solo de gestionar las comunicaciones del sistema distribuido, sino también de gobernar la dinámica de la planta en tiempo real.

5. Planificación

El desarrollo de este trabajo se ha dividido en dos grandes fases, marcadas por el contexto académico en el que se ha realizado. La primera parte del proyecto se realizó entre febrero y julio durante mi estancia en la universidad Politechnika Poznańska (Polonia), mientras que la segunda parte (centrada en la simulación, el análisis teórico y la redacción del documento) fue realizada entre los meses de septiembre y enero.

En la siguiente tabla se resume la planificación real del proyecto, organizada por tareas y periodos:

Fase	Periodo	Tareas principales
Fase 1: Diseño preliminar y selección de hardware	Febrero (en Polonia)	Análisis de requisitos, elección de la placa STM32 NUCLEO-F446ZE, evaluación de compatibilidad con micro-ROS.
Fase 2: Configuración del entorno de desarrollo	Febrero - Marzo (en Polonia)	Instalación y uso de STM32CubeIDE, integración de micro-ROS en el microcontrolador, configuración de FreeRTOS y periféricos.
Fase 3: Comunicación con ROS 2 y UART	Marzo - Abril (en Polonia)	Desarrollo del nodo ROS 2 en Python, suscripción al tópico <code>/darjan/demanded_speed</code> , recepción UART en el STM32.
Fase 4: Control físico del motor	Abril - Mayo (en Polonia)	Uso de Motor Control Workbench para configurar el driver B-G431B-ESC1, transmisión de consignas y activación del motor real.
Fase 5: Validación funcional en laboratorio	Mayo - Junio (en Polonia)	Verificación completa del sistema físico (ROS 2 → STM32 → ESC → motor), observación de respuesta

		del sistema.
Fase 6: Pausa y reactivación	Julio - Septiembre	Fin de la estancia Erasmus y pausa del proyecto hasta septiembre.
Fase 7: Simulación con Simulink y estudio PID	Septiembre - Diciembre (en Barcelona)	Desarrollo de modelo PID en Simulink, comparación entre sistema simulado y real, análisis del comportamiento dinámico.
Fase 8: Redacción y cierre del trabajo	Diciembre - Enero (en Barcelona)	Elaboración de la memoria completa.

Tabla 5.1. Planificación temporal del proyecto por fases y tareas. Fuente: Propia

La posibilidad de trabajar primero con el sistema físico real permitió validar la viabilidad técnica del proyecto. Más adelante, la parte de simulación permitió complementar el análisis con una visión más controlada del comportamiento del sistema.

Este enfoque mixto (laboratorio + simulación) ha resultado muy útil para comparar y analizar el sistema desde una perspectiva práctica y teórica.

6. Estudio económico

Este apartado presenta una estimación económica aproximada del coste asociado a la realización del trabajo de fin de grado. Aunque en este caso el proyecto no ha supuesto un gasto real más allá del tiempo dedicado, se han tenido en cuenta todos los recursos utilizados como si se tratara de un encargo profesional, con el objetivo de valorar de forma realista su alcance.

Coste del trabajo personal

La carga lectiva asignada a un TFG en el plan de estudios es de 12 créditos ECTS, lo que equivale a 300 horas de dedicación total. Aunque no todas estas horas se hayan dedicado al desarrollo práctico, sí que incluyen el trabajo completo realizado: aprendizaje de herramientas, desarrollo experimental, integración de sistemas, pruebas, redacción, etc.

A continuación, se detalla una estimación de la distribución aproximada de horas por tipo de tarea:

Tarea	Horas estimadas
Documentación y lectura preliminar	30 h
Diseño y configuración de hardware	35 h
Configuración de micro-ROS y ROS 2	40 h
Programación en STM32 y pruebas UART	40 h
Integración con driver y control del motor	30 h
Simulación en Simulink	25 h
Redacción de la memoria del TFG	60 h
Depuración, ajustes finales y validación	40 h
Total estimado	300 h

Tabla 6.1. Estimación de la dedicación horaria y coste del trabajo personal. Fuente: Propia

Considerando un valor medio de 15 €/h por hora de trabajo técnico, el coste asociado a la dedicación personal sería:

$$300 \text{ h} \times 15 \text{ €/h} = 4.500 \text{ €}$$

Ecuación (6.1)

Costes operativos indirectos

Aunque no se han producido gastos operativos significativos a nivel individual, es razonable incluir una estimación proporcional del uso de instalaciones y servicios generales:

Concepto	Estimación (€)
Electricidad (uso laboratorio)	15 €
Acceso a red, calefacción	10 €
Material fungible (cables, protoboards, etc.)	10 €
Total costes operativos	35 €

Tabla 6.2. Desglose de costes operativos indirectos. Fuente: Propia

Materiales y equipos utilizados

La mayor parte del equipamiento ha sido facilitado por el laboratorio, pero para el estudio económico se estima su coste de mercado y se considera una amortización proporcional según el uso.

Elemento	Precio estimado (€)	Vida útil (años)	Uso proporcional (meses)	Coste imputado (€)
STM32 Nucleo-F446Z E	25 €	5	3	1,25 €
B-G431B-ESC 1	60 €	5	3	3,00 €

Motor Maxon EC	150 €	10	3	3,75 €
Fuente de alimentación programable	300 €	10	3	7,50 €
Osciloscopio digital	1000 €	10	3	25,00 €
PC de desarrollo	800 €	5	3	40,00 €
Total equipos y materiales				80,50 €

Tabla 6.3. Presupuesto de materiales, equipos y amortización. Fuente: Propia

Licencias y software

Todos los entornos de desarrollo utilizados han sido de código abierto o gratuitos para uso académico:

- STM32CubeIDE: Gratuito (STMicroelectronics)
- Motor Control Workbench: Gratuito
- ROS 2: Código abierto
- micro-ROS: Código abierto
- Simulink: Licencia académica proporcionada por la universidad
- Visualización UART (STM32CubeMonitor, PuTTY): Gratuitos

Coste total en licencias: 0 €

Coste total estimado

Concepto	Importe (€)
Trabajo personal (300 h × 15 €)	4.500,00 €
Costes operativos generales	35,00 €
Amortización de equipos	80,50 €
Licencias y software	0,00 €
Total sin IVA	4.615,50 €
IVA (21 %)	969,26 €
Total con IVA incluido	5.584,76 €

Tabla 6.4. Coste total estimado del proyecto. Fuente: Propia

Discusión económica

El coste total estimado para la realización de este proyecto es de 5.584,76 €. Aunque este valor puede parecer elevado para un trabajo académico, representa el esfuerzo y el tiempo que conlleva desarrollar este trabajo.

En un entorno real, parte de estos costes podrían optimizarse o reducirse mediante reutilización de hardware, entornos compartidos o desarrollo en equipo.

7. Estudio ambiental

Este proyecto, al estar centrado en el desarrollo de un sistema de control embebido y comunicación con ROS 2, no ha implicado procesos industriales ni generación de residuos peligrosos. Sin embargo, como cualquier actividad tecnológica, ha tenido un impacto ambiental asociado principalmente al consumo energético durante las fases de desarrollo, pruebas en laboratorio y uso de equipos informáticos.

Estimación de consumo eléctrico

Durante todo el desarrollo se han utilizado dos ordenadores para programación, simulación y documentación. También se han realizado varias sesiones de prueba con placas electrónicas alimentadas por fuente de laboratorio, además de cierto uso de equipos como osciloscopios.

La estimación aproximada de energía consumida es la siguiente:

Dispositivo / actividad	Potencia media estimada	Tiempo estimado	Consumo total
Ordenador personal (desarrollo, Simulink, redacción)	60 W	250 h	15,0 kWh
Portátil auxiliar (pruebas físicas)	40 W	50 h	2,0 kWh
Fuente de alimentación de laboratorio (motor)	30 W	20 h	0,6 kWh
Osciloscopio digital (verificación UART/PWM)	25 W	10 h	0,25 kWh
Total estimado			17,85 kWh

Tabla 7.1. Impacto Ambiental: Consumo Energético y Emisiones de CO₂. Fuente: Propia

Emisiones asociadas

Según los factores de emisión publicados por el Institut Català d'Energia para el mix

eléctrico medio de España (2022), se estima una media de 0,2 kg de CO₂/kWh para electricidad de red [31].

Aplicando este factor al consumo estimado:

Emisiones totales $\approx 17,85 \text{ kWh} \times 0,2 \text{ kg CO}_2/\text{kWh} = 3,57 \text{ kg CO}_2\text{e}$ Ecuación (7.1)

Este valor es relativamente bajo teniendo en cuenta el alcance del proyecto.

Evaluación crítica

Aunque el impacto generado es bajo, podrían haberse tomado medidas adicionales para reducirlo aún más. Algunas posibles mejoras serían las siguientes:

- Utilizar ordenadores energéticamente eficientes.
- Realizar simulaciones en momentos concentrados.
- Apagar completamente el instrumental de laboratorio cuando no se esté utilizando.

Este tipo de medidas, aunque pequeñas, ayudan a reducir el impacto ambiental de proyectos similares cuando sean desarrollados a gran escala. En el caso de este proyecto no implicaría un cambio demasiado significativo, pero con un proyecto de estas características realizado a una escala mayor, sí que podría llegar a ser diferencial a nivel del impacto ambiental realizado.

8. Estudio social y de igualdad de género

El trabajo desarrollado forma parte del ámbito de la ingeniería electrónica, el control de sistemas embebidos y la comunicación entre dispositivos a través de tecnologías como ROS 2 y micro-ROS. Se trata de un entorno claramente técnico que, a día de hoy, continúa estando mayoritariamente masculinizado. Esta realidad no pienso que sea debida a una discriminación directa derivada de las herramientas o de la propia tecnología, sino a factores sociales y culturales que todavía influyen en la participación equilibrada de distintos géneros en este tipo de ámbitos [27].

A lo largo del proyecto no se ha producido ningún tipo de restricción en el acceso a la tecnología por razones de género, identidad o capacidad económica. Las herramientas utilizadas son de acceso público o disponen de licencias académicas que no establecen ningún tipo de discriminación de género. Dicho esto, sí es cierto que el conocimiento técnico necesario para utilizarlas de forma correcta no siempre está al alcance de toda la población, especialmente de aquellas personas que no han tenido la oportunidad de tener acceso a una formación técnica previa.

Por otro lado, el sistema desarrollado no plantea diferencias de impacto ni de aplicación en función del género. Se trata de una solución de carácter general, diseñada para resolver un problema concreto de comunicación y control, sin proporcionar elementos que puedan generar discriminación directa o indirecta hacia hombres, mujeres o personas con otras identidades de género. El enfoque que se ha empleado ha sido plenamente funcional y técnico.

En cuanto al las personas involucradas en el desarrollo del trabajo, este ha estado formado por una única persona bajo la supervisión de un único tutor académico. Esto no permite realizar un análisis válido a nivel de equilibrio de género dentro del equipo , aunque sí que puede llegar a representar una realidad: la representación femenina en algunos entornos técnicos y académicos sigue siendo reducida, y es un aspecto que debe seguir mejorándose con el tiempo.

Durante la redacción de este documento se ha tratado de utilizar un lenguaje técnico, evitando expresiones sexistas o excluyentes. Además, no se han utilizado imágenes, ejemplos ni referencias que se puedan relacionar con discriminaciones culturales, de género o hacia algún colectivo en concreto.

En definitiva, el proyecto ha sido desarrollado desde una perspectiva de neutralidad, respeto y conciencia de la desigualdad todavía existente en entornos técnicos de ingeniería como este.

9. Conclusiones

La realización de este proyecto ha permitido validar la viabilidad de integrar microcontroladores de bajo coste dentro de arquitecturas robóticas distribuidas cumpliendo con los objetivos que se han planteado en un inicio. A continuación, se detallan las principales conclusiones extraídas del desarrollo del trabajo:

En primer lugar, se ha demostrado la eficacia y el correcto funcionamiento de micro-ROS como middleware para comunicar el entorno de ROS 2 a sistemas embebidos. La implementación del agente y el cliente sobre el microcontrolador STM32 ha permitido establecer una comunicación eficaz. Se ha verificado que la estabilidad de la comunicación UART es suficiente para aplicaciones de control de motores en tiempo real, ya que se ha generado un sistema de comunicación efectivo entre los diferentes módulos del sistema distribuido.

En relación al hardware, la elección de la placa B-G431B-ESC1 se ha confirmado como una solución muy válida para este tipo de aplicaciones. Su integración, ha simplificado enormemente el diseño electrónico. Las pruebas experimentales han validado su capacidad para ejecutar algoritmos de conmutación, liberando la carga computacional al nodo de control principal.

Desde el punto de vista del control automático, la obtención experimental de la planta y su posterior modelado en Simulink han sido determinantes. El diseño del controlador PI usando la técnica de cancelación de polos ha permitido obtener una respuesta dinámica estable y sin sobreimpulso, ajustada a las necesidades mecánicas del motor BLDC. Las simulaciones han validado que el sistema es capaz de seguir las referencias de velocidad con precisión, validando teóricamente el algoritmo que se ha propuesto.

Finalmente, este trabajo abre una vía clara para futuros avances: la implementación física del PID en el sistema real. Dado que el controlador ya ha sido validado en simulación, sería muy interesante continuar el proyecto trasladando este algoritmo al firmware del microcontrolador STM32. Esto permitiría cerrar el lazo de velocidad usando sensores externos y controlar de forma total la dinámica del motor desde el sistema embebido, completando de esta forma la autonomía del nodo de actuación. Además el hecho de que el diseño del sistema sea distribuido, proporciona muchísimas posibilidades de control a través de ROS 2, siendo posible añadir variantes al sistema sin modificar su estructura general.

10. Agradecimientos

Me gustaría agradecer a mi directora, Alba Pérez Gracia, por su ayuda y guía a lo largo del desarrollo de el trabajo. Su apoyo ha sido fundamental para enfocar correctamente los objetivos del proyecto y estructurar esta memoria.

También agradezco a la Politechnika Poznanska (Universidad Tecnológica de Poznan) por haberme acogido durante mi estancia académica. Agradezco especialmente la disposición de los recursos materiales y el acceso a las instalaciones y laboratorios, sin los cuales no habría sido posible realizar la fase experimental de este Trabajo de Fin de Grado que tanto esfuerzo me ha llevado.

11. Bibliografía

- [1] Chesi, S., Perez, O., & Romano, M. (2015). A Dynamic, Hardware-in-the-Loop, Three-Axis Simulator of Spacecraft Attitude Maneuvering with Nanosatellite Dimensions. Faculty And Researcher Publications, 4(1), 315-328.
- [2] Open Robotics. (s.f.). ROS 2 Documentation: Humble documentation. <https://docs.ros.org/en/humble/>
- [3] Object Management Group (OMG). (s.f.). Data Distribution Service (DDS) Specification. <https://www.omg.org/spec/DDS/>
- [4] micro-ROS Team. (s.f.). micro-ROS Documentation and Architecture. <https://micro.ros.org/>
- [5] Perucha Martínez, A. (2022). Integración de micro-ROS en una arquitectura robótica basada en ROS 2 (Trabajo de Fin de Grado). Universidad Politécnica de Cartagena.
- [6] eProxima. (s. f.). Micro XRCE-DDS: Middleware for resource-constrained devices. <https://micro-xrce-dds.docs.eprosima.com/>
- [7] Yedamale, P. (2003). Brushless DC (BLDC) motor fundamentals (Application Note AN885). Microchip Technology Inc. <https://ww1.microchip.com/downloads/en/appnotes/00885a.pdf>
- [8] Zhao, J., & Yu, Y. (2011). Brushless DC motor fundamentals (Application Note AN047). Monolithic Power Systems. https://media.monolithicpower.com/document/Brushless_DC_Motor_Fundamentals.pdf
- [9] Hughes, A., & Drury, B. (2019). Electric Motors and Drives: Fundamentals, Types and Applications. Elsevier.
- [10] De la Fuente Prieto, Á. (2018). Diseño y control de un motor Brushless DC (Trabajo de Fin de Grado). Universidad de Valladolid. <https://uvadoc.uva.es/bitstream/handle/10324/30524/TFG-P-787.pdf>
- [11] Jiménez Hervás, A. (2023). Control vectorial de motor BLDC para simulador de inserción de agujas hipodérmicas (Trabajo Fin de Grado). Universidad Politécnica de Madrid.
- [12] STMicroelectronics. (2019). AN1930: Field Oriented Control (FOC) for 3-phase motors. <https://www.st.com/en/applications/industrial-motor-control/3-phase-field-oriented-contr>

[ol-foc.html](#)

[13] MathWorks. (s.f.). Control System Tuner: Pole Placement. <https://es.mathworks.com/help/control/ref/place.html>

[14] MathWorks. (s.f.). Simscape Electrical: Brushless DC Motor Model. <https://es.mathworks.com/help/sps/ug/brushless-dc-motor.html>

[15] MathWorks. (s.f.). Motor Control Blockset: BLDC Motor Control with Simulink. <https://es.mathworks.com/discovery/bldc-motor-control.html>

[16] STMicroelectronics. (2023). STM32F446 Reference Manual (RM0390). https://www.st.com/resource/en/reference_manual/dm00135183.pdf

[17] STMicroelectronics. (2020). UM2516: Electronic speed controller discovery kit for drones (B-G431B-ESC1). https://www.st.com/resource/en/user_manual/um2516-electronic-speed-controller-discovery-kit-for-drones-with-stm32g431cb-stmicroelectronics.pdf

[18] STMicroelectronics. (s. f.). STM32 Ecosystem for Motor Control. https://www.st.com/content/st_com/en/ecosystems/stm32-motor-control-ecosystem.html !

[19] STMicroelectronics. (2022). How to use STM32 motor control SDK v6.0 workbench (User Manual UM3027).

[20] STMicroelectronics. (2017). UM1727: STM32CubeF4 HAL and LL drivers. https://www.st.com/resource/en/user_manual/um1725-description-of-stm32f4-hal-and-ll-driver-drivers-stmicroelectronics.pdf

[21] STMicroelectronics. (2018). AN2584: UART communication using DMA. https://www.st.com/resource/en/application_note/an2548-introduction-to-dma-controller-for-stm32-mcus-stmicroelectronics.pdf

[22] Texas Instruments. (2022). UART protocol and error overview. <https://www.ti.com/>

[23] Maxon Motor. (2020). EC 45 flat 50 Watt (251601) Operating Instructions. https://www.maxongroup.es/medias/sys_master/root/9245475536926/Manual-EC-45-FLAT-IE-EN-2023-12.pdf

[24] FreeRTOS. (s.f.). FreeRTOS Kernel Documentation. https://www.freertos.org/Documentation/RTOS_book.html

[25] eProxima. (s.f.). Fast DDS Documentation. <https://fast-dds.docs.eprosima.com/>

-
- [26] Barr, M., & Massa, A. (2006). Programming Embedded Systems, with C and GNU Development Tools (2nd ed.). O'Reilly Media. Recuperado de https://www.bogotobogo.com/cplusplus/files/embed/OReilly_Programming_Embedded_Systems_Second_edition_ebook.pdf
- [27] European Institute for Gender Equality (EIGE). (2018). Gender equality and digitalization.
- [28] Universitat Politècnica de Catalunya. (s.f.). Simulación y Control para Motores Brushless. UPCommons. Recuperado de <https://upcommons.upc.edu/bitstreams/ac6a2e19-59bf-4d26-8bf7-aea556b50101/download>
- [29] Texas Instruments. (1998). Field-Oriented Control of a Three-Phase Synchronous Motor (Application Report BPRA048). Recuperado de <https://www.ti.com/lit/an/bpra048/bpra048.pdf>
- [30] Fundació SERGI. (2022, 12 diciembre). Objectius de desenvolupament sostenible. Recuperado de <https://www.fundaciosergi.org/objectius-de-desenvolupament-sostenible/>
- [31] Generalitat de Catalunya. (2024). Factores de emisión asociados a la energía: Mix eléctrico y combustibles. Oficina Catalana del Canvi Climàtic. Recuperado de https://canviclimatic.gencat.cat/es/actua/factors_demissio_associats_a_lenergia/